

Exercises in pills

Bruno Iannone

October 2025

1 Discrete filtering: Localization

We want to estimate the **current state belief** given:

- A discrete map (a grid).
- **Previous state belief**
- Sequence of observations $\vec{z}_{0:t}$
- Sequence of controls $\vec{u}_{0:t-1}$
- **Transition model**
- **Observation model**

Bayesian filter formula

$$b(\vec{x}_t) = \eta_t p(\vec{z}_t | \vec{x}_t) \sum_{\vec{x}_{t-1}} p(\vec{x}_t | \vec{x}_{t-1}, \vec{u}_{t-1}) b(\vec{x}_{t-1})$$

1.1 State

In the following we assume that state is 2 dimensional

$$\vec{x}_t = \begin{pmatrix} x_t \\ y_t \end{pmatrix}$$

Basically map rows and columns.

1.2 Deterministic controls

If controls are deterministic, we are perfectly sure where our robot will be after issuing the control. Therefore, if the movement is feasible, the transition issued by the control will have probability 1. Figure 1.

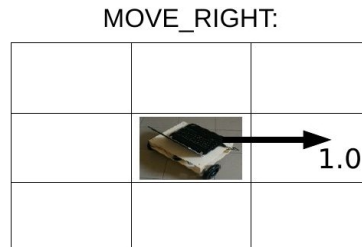


Figure 1: Deterministic transition function

In the deterministic case, the robot can only move on adjacent cells up, down, left or right (no diagonal movements) $\rightarrow$ transitions for further cells have probability 0 (Figure 2).

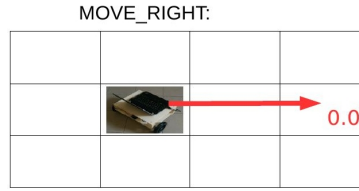


Figure 2: Non adjacent motion transition function

Note: The program logic doesn't allow for the robot to go on an occupied cell. In this case, it will remain where it is with probability 1. Figure 3

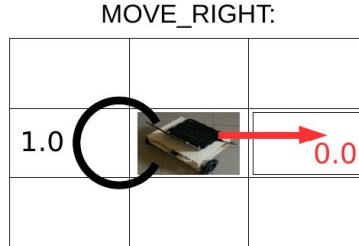


Figure 3: No motion deterministic

1.3 Non deterministic controls

If controls are **non** deterministic, we are **not** perfectly sure where our robot will be after issuing the control. Controls are the same as in the deterministic case, but the robot can move diagonally because it may slips with a certain probability. Therefore, if the movement is feasible, the transition model will have a probability distribution (Figure 4).

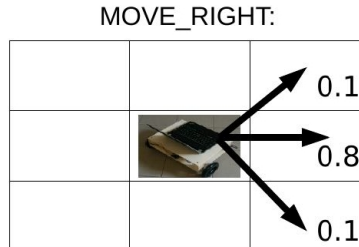


Figure 4: Non deterministic transition function

Moreover, the program logic doesn't allow for the robot to go on an occupied cell. In this case, it will remain where it is with probability 1. Figure 5

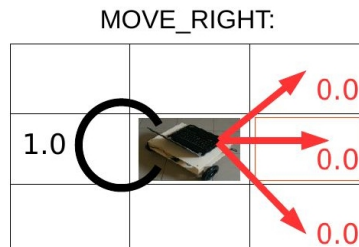


Figure 5: No motion non deterministic

1.3.1 Observation model

In a non deterministic setting, the only way to recover from error is through observations. However, also sensors are wrong with a certain probability and we will model this uncertainty.

1.4 Code

- Initialize belief

```
1 belief_initial_value = 1/(number_of_cells);
2
3 state_belief = ones(rows(map), columns(map))*belief_initial_value;
```

- Transition model - DETERMINISTIC

```
1 function transition_probability_matrix = transitionModel(map, row_from,
2   col_from, control_input)
3
4   transition_probability_matrix = zeros(map.rows, map.cols);
5   [row_to, col_to] = compute_motion(map, row_from, col_from, control_input
6   )
7   if(!is_motion_valid(row_from, col_from, row_to, col_to))
8       return
9
10  transition_probability_matrix(row_to, col_to) = 1;
11 endfunction
```

- Transition model - NON DETERMINISTIC

```
1 #Retrieve real robot position after moving
2 function transition_probability_matrix = transitionModel(map, row_from,
3   col_from, control_input)
4
5   transition_probability_matrix = zeros(map.rows, map.cols);
6   [row_to, col_to] = compute_motion(map, row_from, col_from, control_input
7   )
8   if(!is_motion_valid(row_from, col_from, row_to, col_to))
9       return
10
11  transition_probability_matrix(row_to, col_to) = 0.8;
12  transition_probability_matrix(diagonal_left) = 0.1;
13  transition_probability_matrix(diagonal_right) = 0.1;
14 endfunction
```

- Observation model - NON DETERMINISTIC

```
1 function observation_probability = observationModel(map, row, col,
2   observations)
3
4   observation_probability = 1;
5
6   [bumpers_values] = sense_obstacles(map, row, col)
7
8   for i=1:num_bumpers
9
10      if(bumpers_values[i] == observations[i])
11          observation_probability *=0.8;
12      else
13          #Wrong measurement probability
14          observation_probability *= 0.2;
15
16 endfunction
```

Note: There is only the non deterministic version because you do not need observations if everything is deterministic ☺

At each robot motion:

- Advance the simulation

```

1 #from actual position, retrieve new robot position according to our
   transition model
2 state_ground_truth = getNextState(map, state_ground_truth, control_input);
3
4 #obtain current observations according to our observation model
5 observations = getObservations(map, state_ground_truth);

```

- Predict

```

1 #PREDICT robot position belief
2 state_belief_previous = state_belief;
3 state_belief = zeros(map_rows, map_cols);
4
5 for row = 1:map_rows
6     for col = 1:map_cols
7         state_belief += transitionModel(map, row, col, control_input)*
           state_belief_previous(row, col);
8     endfor
9 endfor

```

- Update

```

1 #UPDATE robot position belief and COMPUTE the normalizer
2 inverse_normalizer = 0;
3
4 for row = 1:map_rows
5     for col = 1:map_cols
6         state_belief(row, col) *= observationModel(map, row, col,
           observations);
7         inverse_normalizer += state_belief(row, col);
8     endfor
9 endfor
10
11 #NORMALIZE the belief probabilities to [0, 1]
12 normalizer = 1/inverse_normalizer;
13 state_belief *= normalizer;

```

1.5 Localization with orientation

1.5.1 State

In the following we assume that state is 3 dimensional

$$\vec{x}_t = \begin{pmatrix} x_t \\ y_t \\ \theta_t \end{pmatrix}$$

Basically map rows, columns and orientation. With $\theta_t \in [0, \pi]$

1.5.2 Deterministic controls

Same as 1.2, but now we also have a rotation left or right. Each rotation is of $\pm \frac{\pi}{2}$. Figure 6

Note: We only assume deterministic controls.

ROTATE_LEFT:

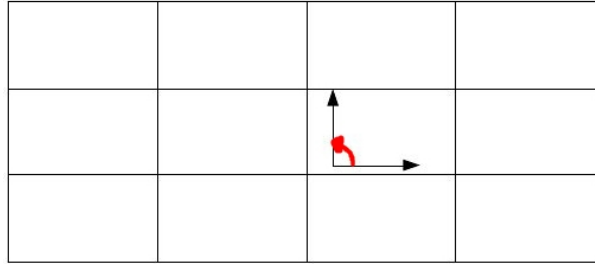


Figure 6: Rotate left example (Rotate right is symmetric)

1.5.3 Observation model

Depending on the orientation of the robot, the bumpers capture different cells on the map. Namely, bumper 1 is in the cell up with orientation 0, but in cell left with orientation $\pi/2$. Figure 7.

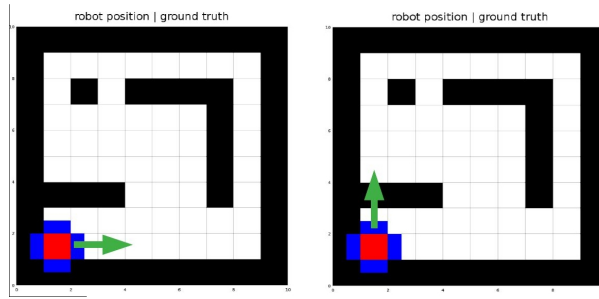


Figure 7: Bumpers orientation change example

We can assume the robot to always face north and just swap the bumpers accordingly to the rotation. This can be done by defining a mapping for each theta value. This mapping is read by column, so column i refers to bumper i and so on. Bumpers position after the rotation can be found by adding to the robot position the entries of each column.

Each column has the following meaning

$$\begin{pmatrix} bumper_{i,row} \\ bumper_{i,col} \end{pmatrix}$$

Hence we take each column, we sum it to the robot position and we find the new bumper location.

```

1 orientation_to_cells_mapping(:, :, 1) = [ -1  1  0  0;
                                           0  0 -1  1];
2
3 orientation_to_cells_mapping(:, :, 2) = [  0  0  1 -1;
                                           -1  1  0  0];
4
5
6 orientation_to_cells_mapping(:, :, 3) = [  1 -1  0  0;
                                           0  0  1 -1];
7
8 orientation_to_cells_mapping(:, :, 4) = [  0  0 -1  1;
                                           1 -1  0  0];
9

```

1.5.4 Code

- Initialize belief

```

1 #THETA_VALUES = 4 (0,pi/2,pi,-pi/2)
2 belief_initial_value = 1/(number_of_free_cells*THETA_VALUES);
3
4 state_belief = ones(rows(map), columns(map), THETA_VALUES);

```

```

5
6 for (theta = 1:THETA_VALUES)
7     state_belief(:, :, theta) *= belief_initial_value;
8 endfor

```

- Transition model - **DETERMINISTIC**

```

1 function transition_probability_matrix = transitionModel(map, row_from,
2   col_from, theta_from, control_input)
3
4   transition_probability_matrix = zeros(map.rows, map.cols, THETA_VALUES)
5   ;
6   [row_to, col_to, theta_to] = compute_motion(map, row_from, col_from,
7     theta_from, control_input)
8   if(!is_motion_valid(row_to, col_to, theta_to, row_from, col_from,
9     theta_from)
10       return
11
12   transition_probability_matrix(row_to, col_to, theta_to) = 1;
13 endfunction

```

- Observation model - **NON DETERMINISTIC**

```

1 function observation_probability = observationModel(map, row, col, theta,
2   observations)
3
4   observation_probability = 1;
5
6   rotate_bumpers(row, col, theta)
7   [bumpers_values] = sense_obstacles(map, row, col, theta)
8
9   for i=1:num_bumpers
10
11       if(bumpers_values[i] == observations[i])
12           observation_probability *=0.8;
13       else
14           observation_probability *= 0.2;
15
16   endfunction

```

Note: There is only the non deterministic version because you do not need observations if everything is deterministic ☺

At each robot motion:

- Advance the simulation

```

1 #from actual position, retrieve new robot position according to our
2   transition model
3 state_ground_truth = getNextState(map, state_ground_truth, control_input);
4
5 #obtain current observations according to our observation model
6 observations = getObservations(map, state_ground_truth);

```

- Predict

```

1 #PREDICT robot position belief
2 state_belief_previous = state_belief;
3 state_belief = zeros(map_rows, map_cols, THETA_VALUES);
4
5 for (theta=1:THETA_VALUES)
6     for row = 1:map_rows
7         for col = 1:map_cols

```

```

8         state_belief += transitionModel(map, row, col, theta control_input)
          *state_belief_previous(row, col, theta);
9     endfor
10 endfor
11 endfor

```

- Update

```

1 #UPDATE robot position belief and COMPUTE the normalizer
2 inverse_normalizer = 0;
3
4 for (theta=1:THETA_VALUES)
5     for row = 1:map_rows
6         for col = 1:map_cols
7             state_belief(row, col, theta) *= observationModel(map, row, col
8                 , theta, observations);
9             inverse_normalizer += state_belief(row, col, theta);
10        endfor
11    endfor
12 endfor
13 #NORMALIZE the belief probabilities to [0, 1]
14 normalizer = 1/inverse_normalizer;
15 state_belief *= normalizer;

```

2 EKF localization

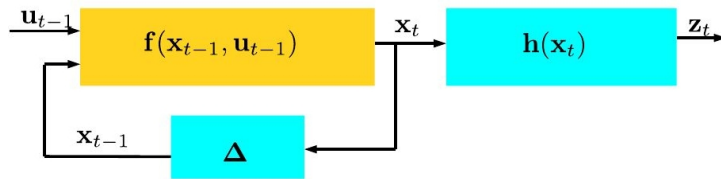


Figure 8: Dynamic schema

We want to estimate the **current state distribution** given:

- Map as a 2D plane
- **Previous sate distribution**
- Sequence of observations $\vec{z}_{0:t}$
- Sequence of controls $\vec{u}_{0:t-1}$
- **Transition model**
- **Observation model**

EKF formula

- Predict

$$\vec{\mu}_{t|t-1} = \mathbf{f} \left(\vec{\mu}_{t-1|t-1}, \vec{u}_{t-1} \right)$$

$$\Sigma_{t|t-1} = A_t \Sigma_{t-1|t-1} A_t^T + B_t \Sigma_u B_t^T$$

- Update

$$\vec{\mu}_z = \mathbf{h}(\vec{\mu}_{t|t-1})$$

$$\vec{\mu}_{t|t} = \vec{\mu}_{t|t-1} + K_t (\vec{z}_t - \vec{\mu}_z)$$

$$\Sigma_{t|t} = (I - K_t C_t) \Sigma_{t|t-1}$$

2.1 Assumptions

In the following we assume that

- Robot is controlled by translational and rotational velocities
- Senses a set of **uniquely distinguishable landmarks** through a “2D landmark sensors”
- The **location** $l^{[i]}$ of the landmarks w.r.t. the world is **known**.

$$^W l^{[i]} = \begin{pmatrix} x^{[i]} \\ y^{[i]} \end{pmatrix} \in \mathbb{R}^2$$

2.2 Domains

- State space (isometry)

$$\mathbf{X}_t = [\mathbf{R}_t | \vec{t}_t] \in SE(2) \rightarrow \vec{x}_t = \begin{pmatrix} x_t \\ y_t \\ \theta_t \end{pmatrix} \in \mathbb{R}^3$$

Note: Poses are not euclidean but we map them to 3D vectors

- Space of controls (inputs)

$$\vec{u}_t = \begin{pmatrix} \Delta_t v_t \\ \Delta_t \omega_t \end{pmatrix} = \begin{pmatrix} u_t^1 \\ u_t^2 \end{pmatrix} \in \mathbb{R}^2$$

- Space of observations (measurements)

$$\vec{z}_t^{[i]} = \begin{pmatrix} x_t^{[i]} \\ y_t^{[i]} \end{pmatrix} \in \mathbb{R}^2$$

2.3 Transition function

Considering constant velocity in the time interval $[t_{t-1}|t_t]$, we can obtain $\vec{x}_t$ by Euler integration

$$\vec{x}_t = \mathbf{f}(\vec{x}_{t-1}, \vec{u}_{t-1}) = \begin{pmatrix} x_{t-1} + u_{t-1}^1 \cos(\theta_{t-1}) \\ y_{t-1} + u_{t-1}^1 \sin(\theta_{t-1}) \\ \theta_{t-1} + u_{t-1}^2 \end{pmatrix}$$

Note: $\vec{\mu}_{t|t-1} = \vec{x}_t$

We want a linear model, so we compute the jacobians to linearize it using Taylor expansion

$$\mathbf{A}_t = \frac{\partial \mathbf{f}(\cdot)}{\partial \vec{x}} = \begin{pmatrix} 1 & 0 & -u_{t-1}^1 \sin(\theta_{t-1}) \\ 0 & 1 & u_{t-1}^1 \cos(\theta_{t-1}) \\ 0 & 0 & 1 \end{pmatrix}$$

$$\mathbf{B}_t = \frac{\partial \mathbf{f}(\cdot)}{\partial \vec{u}} = \begin{pmatrix} \cos(\theta_{t-1}) & 0 \\ \sin(\theta_{t-1}) & 0 \\ 0 & 1 \end{pmatrix}$$

Note: $\mathbf{A}_t$ is a $N \times N$ matrix, $\mathbf{B}_t$ is a $N \times U$ (U = control size)

In this example $N = 3$ and $U = 2$. Therefore $\mathbf{A}_t$ is a 3×3 while $\mathbf{B}_t$ is a 3×2

2.3.1 Control Noise

We assume that the velocity measurements are affected by a Gaussian noise resulting from the sum of a term with constant standard deviation (σ) and a velocity dependent term (u) whose standard deviation grows with the speed.

Translational and rotational noise are assumed independent

$$\mathbf{n}_{u,t} \sim \mathcal{N}\left(\mathbf{n}_{u,t}; \mathbf{0}, \begin{bmatrix} \sigma_v^2 + (u_t^1)^2 & 0 \\ 0 & \sigma_\omega^2 + (u_t^2)^2 \end{bmatrix}\right)$$

2.4 Measurement function

Each measurement $\vec{z}_t^{[i]}$ is the i -th measurement¹. One per landmark.

$$\vec{z}_t^{[i]} = \mathbf{h}^{[i]}(\vec{x}_t) = \mathbf{R}_t^T \left(\vec{l}^{[i]} - \vec{t}_t \right) = \begin{pmatrix} \cos \theta_t (x_t^{[i]} - x_t) + \sin \theta_t (y_t^{[i]} - y_t) \\ -\sin \theta_t (x_t^{[i]} - x_t) + \cos \theta_t (y_t^{[i]} - y_t) \end{pmatrix}$$

Where ${}^w\mathbf{R}_{r,t} = \mathbf{R}_t = \begin{pmatrix} \cos \theta_t & -\sin \theta_t \\ \sin \theta_t & \cos \theta_t \end{pmatrix}$ is the robot w.r.t. the world frame rotation matrix at time t .

At each point in time, our robot will sense only a subset of $\bar{K}$ landmarks in the map. The measurement is thus consisting of a stack of measurements

$$\vec{z}_t = \mathbf{h}(\vec{x}_t) = \begin{pmatrix} h^{[i_1]}(\vec{x}_t) \\ h^{[i_2]}(\vec{x}_t) \\ \vdots \\ h^{[i_{\bar{K}}]}(\vec{x}_t) \end{pmatrix} = \begin{pmatrix} z_t^{[i_1]} \\ z_t^{[i_2]} \\ \vdots \\ z_t^{[i_{\bar{K}}]} \end{pmatrix}$$

Which is to be read as *the i -th measurement produced by the k -th landmark*.

We want a linear model, so we compute the jacobians to linearize it using Taylor expansion

$$\mathbf{C}_t^{[i]} = \frac{\partial \mathbf{h}^{[i]}(\cdot)}{\partial \vec{x}} = \begin{pmatrix} -\mathbf{R}_t^T & \frac{\partial \mathbf{R}_t^T}{\partial \theta_t} \left(\vec{l}^{[i]} - \vec{t}_t \right) \end{pmatrix}$$

Where

$$\frac{\partial \mathbf{R}_t}{\partial \theta_t} = \begin{pmatrix} -\sin \theta_t & -\cos \theta_t \\ \cos \theta_t & -\sin \theta_t \end{pmatrix}$$

Therefore

$$\mathbf{C}_t = \frac{\partial \mathbf{h}}{\partial \vec{x}} = \begin{pmatrix} \frac{\partial \mathbf{h}^{[i_1]}}{\partial \vec{x}} \\ \frac{\partial \mathbf{h}^{[i_2]}}{\partial \vec{x}} \\ \vdots \\ \frac{\partial \mathbf{h}^{[i_{\bar{K}}]}}{\partial \vec{x}} \end{pmatrix} = \begin{pmatrix} C_t^{[i_1]} \\ C_t^{[i_2]} \\ \vdots \\ C_t^{[i_{\bar{K}}]} \end{pmatrix}$$

Note: Each $\frac{\partial \mathbf{h}^{[i_k]}}{\partial \vec{x}}$ is a $Z \times N$ matrix (Z = measurement size). Therefore $\mathbf{C}_t$ is a $(\bar{K} \cdot Z) \times N$ matrix.

In this example $Z = 2$, $N = 3$ hence $\mathbf{C}_t$ is $(\bar{K} \cdot 2) \times 3$

2.4.1 Measurement Noise

We assume it has zero mean with constant standard deviation.

Noise affecting the x and y components of the landmark position are assumed to be independent

$$\mathbf{n}_z \sim \mathcal{N}\left(\mathbf{n}_z; \mathbf{0}, \begin{bmatrix} \sigma_z^2 & 0 \\ 0 & \sigma_z^2 \end{bmatrix}\right)$$

Σ_z will be a $(2\bar{K}) \times (2\bar{K})$ matrix

¹Remember, this is the **predicted** measurement

2.5 Code

- Init values

```
1 [mu, sigma] = init()
```

- Predict

```
1 function [mu, sigma] = prediction(mu, sigma, u)
2
3     %predict mu, this is our f(x,u)
4
5     mu = transition_model(mu, u);
6
7     %Jacobian A
8     A = [ 1, 0, -u.x*sin(mu.theta);
9           0, 1,  u.x*cos(mu.theta);
10          0, 0, 1];
11
12     %Jacobian B
13     B = [ cos(mu.theta) 0;
14          sin(mu.theta) 0;
15          0 1];
16
17     sigma_u = [ noise^2 + u.x^2, 0;
18                0, noise^2 + u.w^2];
19
20     %predict sigma
21     sigma = A*sigma*A' + B*sigma_u*B';
22
23 end
```

- Update

```
1 function [mu, sigma] = correction(mu, sigma, landmarks, observations)
2
3     % determine how many landmarks we have seen
4     num_landmarks_seen = length(observations);
5
6     % dimension of the state in dim, in our case is fixed to 3
7     state_dim = size(mu,1);
8
9     %if I've seen no landmarks, i do nothing
10    if (num_landmarks_seen==0)
11        return;
12    endif
13
14    #transposed rotation matrix
15    Rt=[cos(mu.theta), sin(mu.theta);
16        -sin(mu.theta), cos(mu.theta)];
17
18    # derivative of transposed rotation matrix
19    Rtp=[-sin (mu.theta),cos(mu.theta);
20         -cos(mu.theta), -sin(mu.theta)];
21
22    # here in one go, we go through all landmark measurements vector
23    # for each landmark, we assemble
24    # the "total" measurement vector, containing all stacked measures
25    # the "total" prediction vector, containing all staked predictions
26    # the "total" jacobian, consisting of all jacobians of predictions
27    stacked
28
29    z_t = zeros(0, 1);
30    h_t = zeros(0, 1);
```

```

30 C_t = zeros(0, 1);
31
32 for i=1:num_landmarks_seen
33     %retrieve info about the observed landmark
34     measurement = observations(i);
35
36     % stack current measurement into our meas vec
37     z_t = [z_t; measurement];
38
39     [land.x, land.y] = searchById(landmarks, measurement.id);
40     l = [land.x, land.y]
41     t=[mu.x; mu.y];
42     measure_prediction = Rt*(l-t);
43
44     % current landmark observation
45     h_t = [h_t; measure_prediction]; % stack current observation into our
        obs vec
46
47     %compute its Jacobian
48     C=zeros(2,state_dim);
49     C(1:2,1:2)=-Rt;
50     C(1:2,state_dim)=Rtp*(l-t);
51     C_t = [C_t; C];
52 endfor
53
54 sigma_z = eye(2*num_landmarks_seen)*noise;
55 %Kalman gain
56 K = sigma*C_t'*(inv(C_t*sigma*C_t' + sigma_z));
57 %update mu
58 error = (z_t - h_t);
59 correction = K*error;
60 mu += correction;
61
62 %update sigma
63 sigma *= (eye(state_dim) - K*C_t);
64 end

```

- Transition function

```

1 #Compute mu at time t given mu at time t-1
2 function mu_prime = transition_model(mu, u)
3     mu_prime(1) = mu.x + u.x*cos(mu.theta);
4     mu_prime(2) = mu.y + u.x*sin(mu.theta);
5     mu_prime(3) = mu.theta + u.w;
6 end

```

- At each simulation step

```

1 #Compute mu at time t given mu at time t-1
2 while(true)
3
4     %predict
5     [mu, sigma] = prediction(mu, sigma, u);
6
7     observations = getObservation(mu, sigma)
8
9     %correct
10    [mu, sigma] = correction(mu, sigma, landmarks, observations);
11
12 endfor

```

2.6 EKF localization bearing only

2.6.1 Assumptions

In the following we assume that

- Robot is controlled by translational and rotational velocities
- Senses a set of **uniquely distinguishable landmarks** through a “2D bearing only landmark sensors”
- The **location** $l^{[i]}$ of the landmarks w.r.t. the world is **known**.

$$w_{l^{[i]}} = \begin{pmatrix} x^{[i]} \\ y^{[i]} \end{pmatrix} \in \mathbb{R}^2$$

2.6.2 Domains

- State space (isometry)

$$\mathbf{X}_t = [\mathbf{R}_t | \vec{t}_t] \in SE(2) \rightarrow \vec{x}_t = \begin{pmatrix} x_t \\ y_t \\ \theta_t \end{pmatrix} \in \mathbb{R}^3$$

Note: Poses are not euclidean but we map them to 3D vectors

- Space of controls (inputs)

$$\vec{u}_t = \begin{pmatrix} \Delta_t v_t \\ \Delta_t \omega_t \end{pmatrix} = \begin{pmatrix} u_t^1 \\ u_t^2 \end{pmatrix} \in \mathbb{R}^2$$

- Space of observations (measurements)

$$\vec{z}_t \in SO(2) \rightarrow \mathbb{R}$$

2.6.3 Transition function & Control noise

As 2.3 and 2.3.1

2.6.4 Measurement function

Each measurement $\vec{z}_t^{[i]}$ is the i -th measurement. One per landmark.

$$\vec{z}_t^{[i]} = \mathbf{h}^{[i]}(\vec{x}_t) = \text{atan}_2 \left(\frac{\hat{y}_t^{[i]}}{\hat{x}_t^{[i]}} \right)$$

Where

$$\hat{p}_t^{[i]} = \begin{pmatrix} \hat{x}_t^{[i]} \\ \hat{y}_t^{[i]} \end{pmatrix} = \mathbf{R}_t^T (\vec{l}^{[i]} - \vec{t}_t) = \begin{pmatrix} \cos \theta_t (x^{[i]} - x_t) + \sin \theta_t (y^{[i]} - y_t) \\ -\sin \theta_t (x^{[i]} - x_t) + \cos \theta_t (y^{[i]} - y_t) \end{pmatrix}$$

Where ${}^w\mathbf{R}_{r,t} = \mathbf{R}_t = \begin{pmatrix} \cos \theta_t & -\sin \theta_t \\ \sin \theta_t & \cos \theta_t \end{pmatrix}$ is the robot w.r.t. the world frame rotation matrix at time t .

At each point in time, our robot will sense only a subset of $\bar{K}$ landmarks in the map. The measurement is thus consisting of a stack of measurements

$$\vec{z}_t = \mathbf{h}(\vec{x}_t) = \begin{pmatrix} h^{[i_1]}(\vec{x}_t) \\ h^{[i_2]}(\vec{x}_t) \\ \vdots \\ h^{[I_K]}(\vec{x}_t) \end{pmatrix} = \begin{pmatrix} z_t^{[i_1]} \\ z_t^{[i_2]} \\ \vdots \\ z_t^{[I_K]} \end{pmatrix}$$

Which is to be read as *the i -th measurement produced by the k -th landmark*.

We want a linear model, so we compute the jacobians to linearize it using Taylor expansion

$$\mathbf{h}^{[i]}(\mathbf{x}_t) = \text{atan2}\left(\frac{\hat{y}_t^{[i]}}{\hat{x}_t^{[i]}}\right)$$

$$\hat{\mathbf{p}}_t^{[i]} = \begin{pmatrix} \hat{x}_t^{[i]} \\ \hat{y}_t^{[i]} \end{pmatrix} = \mathbf{R}_t^T (\mathbf{l}^{[i]} - \mathbf{t}_t) = \begin{pmatrix} \cos \theta_t (x^{[i]} - x_t) + \sin \theta_t (y^{[i]} - y_t) \\ -\sin \theta_t (x^{[i]} - x_t) + \cos \theta_t (y^{[i]} - y_t) \end{pmatrix}$$

using multivariate chain rule

$$\frac{\partial \mathbf{h}^{[i]}(\cdot)}{\partial \vec{x}} = \frac{\partial \text{atan2}(\hat{\mathbf{p}}_t^{[i]})}{\partial \hat{\mathbf{p}}_t^{[i]}} \bigg|_{\hat{\mathbf{p}}_t^{[i]} = \mathbf{g}(\vec{x})} \frac{\partial \mathbf{g}_i(\vec{x})}{\partial \vec{x}} \bigg|_{\vec{x} = \vec{x}_i}$$

$$\frac{\partial \text{atan2}(\hat{\mathbf{p}}_t^{[i]})}{\partial \hat{\mathbf{p}}_t^{[i]}} = \frac{1}{1 + \left(\frac{\hat{y}_t^{[i]}}{\hat{x}_t^{[i]}}\right)^2} \begin{pmatrix} -\frac{\hat{y}_t^{[i]}}{(\hat{x}_t^{[i]})^2} & \frac{1}{\hat{x}_t^{[i]}} \end{pmatrix}$$

$$\frac{\partial \mathbf{h}^{[i]}(\cdot)}{\partial \vec{x}} = \mathbf{C}_t^{[i]} = \frac{1}{(\hat{x}_t^{[i]})^2 + (\hat{y}_t^{[i]})^2} \begin{pmatrix} -\hat{y}_t^{[i]} & \hat{x}_t^{[i]} \end{pmatrix} \begin{pmatrix} -\mathbf{R}_t^T \frac{\partial \mathbf{R}_t^T}{\partial \theta_t} \Delta \mathbf{t}^{[i]} \end{pmatrix}$$

Where

$$\Delta \mathbf{t}^{[i]} = \vec{l}^{[i]} - \vec{t}_t$$

Therefore

$$\mathbf{C}_t = \frac{\partial \mathbf{h}}{\partial \vec{x}} = \begin{pmatrix} \frac{\partial \mathbf{h}^{[i_1]}}{\partial \vec{x}} \\ \frac{\partial \mathbf{h}^{[i_2]}}{\partial \vec{x}} \\ \vdots \\ \frac{\partial \mathbf{h}^{[i_{\bar{K}}]}}{\partial \vec{x}} \end{pmatrix} = \begin{pmatrix} \mathbf{C}_t^{[i_1]} \\ \mathbf{C}_t^{[i_2]} \\ \vdots \\ \mathbf{C}_t^{[i_{\bar{K}}]} \end{pmatrix}$$

Note: Each $\frac{\partial \mathbf{h}^{[i_k]}}{\partial \vec{x}}$ is a $Z \times N$ matrix (Z = measurement size). Therefore $\mathbf{C}_t$ is a $(\bar{K} \cdot Z) \times N$ matrix.

In this example $Z = 1$, $N = 3$ hence $\mathbf{C}_t$ is $(\bar{K} \cdot 1) \times 3$

2.6.5 Measurement Noise

For each landmark we measure, we consider a Gaussian noise in the measured angle. We assume it has zero mean with constant standard deviation.

$$\mathbf{n}_z \sim \mathcal{N}(\mathbf{n}_z; \mathbf{0}, [\sigma_z^2])$$

Σ_z will be a diagonal $(\bar{K} \times Z)$ matrix. In this case a $(\bar{K} \times 1)$.

2.6.6 Code

- Init, predict, transition function and simulation update are exactly like 2.5
- Update

```

1 function [mu, sigma] = correction(mu, sigma, landmarks, observations)
2
3 % determine how many landmarks we have seen
4 num_landmarks_seen = length(observations);
5
6 % dimension of the state in dim, in our case is fixed to 3
7 state_dim = size(mu,1);
8
9 %if I've seen no landmarks, i do nothing
10 if (num_landmarks_seen==0)
11     return;

```

```

12     endif
13
14     #transposed rotation matrix
15     Rt=[cos(mu.theta), sin(mu.theta);
16         -sin(mu.theta), cos(mu.theta)];
17
18     # derivative of transposed rotation matrix
19     Rtp=[-sin(mu.theta), cos(mu.theta);
20         -cos(mu.theta), -sin(mu.theta)];
21
22     # here in one go, we go through all landmark measurements vector
23     # for each landmark, we assemble
24     # the "total" measurement vector, containing all stacked measures
25     # the "total" prediction vector, containing all staked predictions
26     # the "total" jacobian, consisting of all jacobians of predictions
27     stacked
28
29     z_t = zeros(0, 1);
30     h_t = zeros(0, 1);
31     C_t = zeros(0, 1);
32
33     for i=1:num_landmarks_seen
34         %retrieve info about the observed landmark
35         measurement = observations(i);
36
37         % stack current measurement into our meas vec
38         z_t = [z_t; measurement.bearing];
39
40         [land.x, land.y] = searchById(landmarks, measurement.id);
41         l = [land.x, land.y]
42         t=[mu.x; mu.y];
43         p_hat = Rt*(l-t);
44
45         bearing_prediction = atan2(p_hat.y, p_hat.x);
46
47         % stack current observation into our obs vec
48         h_t = [h_t; bearing_prediction];
49
50         %compute its Jacobian
51         x_hat = p_hat.x;
52         y_hat = p_hat.y;
53
54         C = 1/(x_hat^2+y_hat^2)*[-y_hat x_hat]*[-Rt Rtp*delta_t];
55         C_t = [C_t; C];
56     endfor
57
58     sigma_z = eye(num_landmarks_seen)*noise;
59     %Kalman gain
60     K = sigma*C_t'*(inv(C_t*sigma*C_t' + sigma_z));
61     %update mu
62     error = (z_t - h_t);
63     correction = K*error;
64     mu += correction;
65
66     %update sigma
67     sigma *= (eye(state_dim) - K*C_t);
68 end

```

3 EKF SLAM

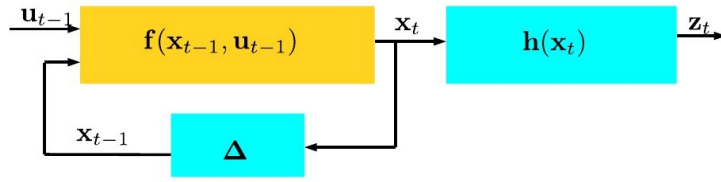


Figure 9: Dynamic schema

We want to estimate the **current state distribution** given:

- Previous state distribution
- Sequence of observations $\vec{z}_{0:t}$
- Sequence of controls $\vec{u}_{0:t-1}$
- Transition model
- Observation model

EKF formula

- Predict

$$\vec{\mu}_{t|t-1} = \mathbf{f} \left(\vec{\mu}_{t-1|t-1}, \vec{u}_{t-1} \right)$$

$$\Sigma_{t|t-1} = A_t \Sigma_{t-1|t-1} A_t^T + B_t \Sigma_u B_t^T$$

- Update

$$\vec{\mu}_z = \mathbf{h}(\vec{\mu}_{t|t-1})$$

$$\vec{\mu}_{t|t} = \vec{\mu}_{t|t-1} + K_t (\vec{z}_t - \vec{\mu}_z)$$

$$\Sigma_{t|t} = (I - K_t C_t) \Sigma_{t|t-1}$$

3.1 Assumptions

In the following we assume that

- Robot is controlled by translational and rotational velocities
- Senses a set of **uniquely distinguishable landmarks** through a “2D landmark sensor”
- The **location** $l^{[i]}$ of the landmarks w.r.t. the world is **unknown**.
- No prior knowledge on the map

Note: We want to develop a KF based algorithm to track the position of the robot as it moves (localization) and, at the same time, the position of the observed landmarks (mapping). Since the landmarks positions are **unknown**, we need to estimate them $\rightarrow$ we put them in the state.

3.2 Domains

- Robot space (isometry)

$$\mathbf{X}_t^{[r]} = [\mathbf{R}_t | \vec{t}_t] \in SE(2) \rightarrow \vec{x}_t^{[r]} = \begin{pmatrix} x_t \\ y_t \\ \theta_t \end{pmatrix} \in \mathbb{R}^3$$

Note: Poses are not euclidean but we map them to 3D vectors

- Space of controls (inputs)

$$\vec{u}_t = \begin{pmatrix} \Delta_t v_t \\ \Delta_t \omega_t \end{pmatrix} = \begin{pmatrix} u_t^1 \\ u_t^2 \end{pmatrix} \in \mathbb{R}^2$$

- Landmarks

$$\vec{x}_t^{[k]} = \begin{pmatrix} x_t^{[k]} \\ y_t^{[k]} \end{pmatrix} \in \mathbb{R}^2$$

With $k = 1, \dots, K$

Remember that in this case K is not fixed, but increases as we move and spot new landmarks.

- Full state vector (robot state + landmarks)

$$\vec{x}_t = \begin{pmatrix} \vec{x}_t^{[r]} \\ \vec{x}_t^{[1]} \\ \vec{x}_t^{[2]} \\ \vdots \\ \vec{x}_t^{[K]} \end{pmatrix} \in \mathbb{R}^{3+2K}$$

Note: 3 is the robot state space dimension. Landmark space has dimension 2. If we measure K landmarks we have size 2K. The total state space dimension is 3+2K

- Space of observations (measurements)

$$\vec{z}_t^{[i]} = \begin{pmatrix} x_t^{[i]} \\ y_t^{[i]} \end{pmatrix} \in \mathbb{R}^2$$

$i = 1, \dots, \bar{K}$

3.3 Transition function

$$\vec{x}_t = \mathbf{f}(\vec{x}_{t-1}, \vec{u}_{t-1}) = \begin{pmatrix} x_{t-1} + u_{t-1}^1 \cos(\theta_{t-1}) \\ y_{t-1} + u_{t-1}^1 \sin(\theta_{t-1}) \\ \theta_{t-1} + u_{t-1}^2 \\ x_{t-1}^{[1]} \\ x_{t-1}^{[2]} \\ \vdots \\ x_{t-1}^{[K]} \end{pmatrix}$$

Note: $\vec{\mu}_{t|t-1} = \vec{x}_t$

Note: Landmarks are still, so they do not evolve in time

We want a linear model, so we compute the jacobians to linearize it using Tailor expansion

$$\mathbf{A}_t = \frac{\partial \mathbf{f}(\cdot)}{\partial \vec{x}} \bigg|_{\vec{x}=\vec{\mu}_t|_{t-1}} = \begin{pmatrix} \frac{\partial \mathbf{f}(\cdot)}{\partial \vec{x}^{[r]}} & \frac{\partial \mathbf{f}(\cdot)}{\partial \vec{x}^{[1]}} & \frac{\partial \mathbf{f}(\cdot)}{\partial \vec{x}^{[2]}} & \dots & \frac{\partial \mathbf{f}(\cdot)}{\partial \vec{x}^{[K]}} \end{pmatrix}$$

Where

•

$$\frac{\partial \mathbf{f}(\cdot)}{\partial \vec{x}^{[r]}} = \begin{pmatrix} 1 & 0 & -u_{t-1}^1 \sin(\theta_{t-1}) \\ 0 & 1 & u_{t-1}^1 \sin(\theta_{t-1}) \\ 0 & 0 & 1 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \\ \vdots & \vdots & \vdots \\ 0 & 0 & 0 \end{pmatrix}$$

•

$$\frac{\partial \mathbf{f}(\cdot)}{\partial \vec{x}^{[1]}} = \begin{pmatrix} 0 & 0 \\ 0 & 0 \\ 0 & 0 \\ 1 & 0 \\ 0 & 1 \\ 0 & 0 \\ \vdots & \vdots \\ 0 & 0 \end{pmatrix}$$

•

$$\frac{\partial \mathbf{f}(\cdot)}{\partial \vec{x}^{[2]}} = \begin{pmatrix} 0 & 0 \\ 0 & 0 \\ 0 & 0 \\ 0 & 0 \\ 0 & 0 \\ 1 & 0 \\ 0 & 1 \\ 0 & 0 \\ \vdots & \vdots \\ 0 & 0 \end{pmatrix}$$

And so on until K

$$\mathbf{B}_t = \frac{\partial \mathbf{f}(\vec{x}, \vec{u})}{\partial \vec{u}} \bigg|_{\vec{u}=\vec{u}_{t-1}} = \begin{pmatrix} \cos(\theta_{t-1}) & 0 \\ \sin(\theta_{t-1}) & 0 \\ 0 & 1 \\ 0 & 0 \\ 0 & 0 \\ \vdots & \vdots \\ 0 & 0 \end{pmatrix}$$

Note: $\mathbf{A}_t$ is a $N \times N$ matrix, $\mathbf{B}_t$ is a $N \times U$ (U = control size)

In this example $N_r = 3$, $N_l = 2$, $U = 2$ and K landmarks. Therefore, $\mathbf{A}_t$ is a $(3 + 2K) \times (3 + 2K)$ while $\mathbf{B}_t$ is a $(3 + 2K) \times 2$

3.3.1 Control Noise

We assume that the velocity measurements are affected by a Gaussian noise resulting from the sum of a term with constant standard deviation (σ) and a velocity dependent term (u) whose standard deviation grows with the speed.

Translational and rotational noise are assumed independent

$$\mathbf{n}_{u,t} \sim \mathcal{N}\left(\mathbf{n}_{u,t}; \mathbf{0}, \begin{bmatrix} \sigma_v^2 + (u_t^1)^2 & 0 \\ 0 & \sigma_\omega^2 + (u_t^2)^2 \end{bmatrix}\right)$$

3.4 Measurement function

Each measurement $\vec{z}_t^{[i]}$ is the i -th measurement. One per landmark.

$$\vec{z}_t^{[i]} = \mathbf{h}^{[i]}(\vec{x}_t^{[r]}, \vec{x}_t^{[i]}) = \mathbf{R}_t^T (\vec{x}_t^{[i]} - \mathbf{t}_t) = \begin{pmatrix} \cos \theta_t (x_t^{[i]} - x_t^{[r]}) + \sin \theta_t (y_t^{[i]} - y_t^{[r]}) \\ -\sin \theta_t (x_t^{[i]} - x_t^{[r]}) + \cos \theta_t (y_t^{[i]} - y_t^{[r]}) \end{pmatrix}$$

Where ${}^w\mathbf{R}_{r,t} = \mathbf{R}_t = \begin{pmatrix} \cos \theta_t & -\sin \theta_t \\ \sin \theta_t & \cos \theta_t \end{pmatrix}$ is the robot w.r.t. the world frame rotation matrix at time t .

At each point in time, our robot will sense only a subset of $\bar{K}$ landmarks in the map.

The measurement is thus consisting of a stack of measurements

$$\vec{z}_t = \mathbf{h}(\vec{x}_t) = \begin{pmatrix} h^{[i_1]}(\vec{x}_t) \\ h^{[i_2]}(\vec{x}_t) \\ \vdots \\ h^{[i_{\bar{K}}]}(\vec{x}_t) \end{pmatrix} = \begin{pmatrix} z_t^{[i_1]} \\ z_t^{[i_2]} \\ \vdots \\ z_t^{[i_{\bar{K}}]} \end{pmatrix}$$

Which is to be read as *the i -th measurement produced by the k -th landmark*.

We want a linear model, so we compute the jacobians to linearize it using Taylor expansion.

$$\mathbf{C}_t^{[i_k]} = \left. \frac{\partial \mathbf{h}^{[i_k]}(\cdot)}{\partial \vec{x}} \right|_{\vec{x}=\vec{\mu}_t|_{t-1}} = \begin{pmatrix} \frac{\partial \mathbf{h}^{[i_k]}(\cdot)}{\partial \vec{x}^{[r]}} & 0 & \dots & \frac{\partial \mathbf{h}^{[i_k]}(\cdot)}{\partial \vec{x}^{[k]}} & \dots & 0 \end{pmatrix}$$

The i -th measurement Jacobian will have a block structure. We need to calculate the derivatives only for the non zero blocks

- Pose block

$$\frac{\partial \mathbf{h}^{[i_k]}(\cdot)}{\partial \vec{x}^{[r]}} = \begin{pmatrix} -\mathbf{R}_t^T & \frac{\partial \mathbf{R}_t^T}{\partial \theta_t} (\vec{x}_t^{[k]} - \vec{t}_t) \end{pmatrix} \rightarrow Z \times N_r = 2 \times 3$$

- Landmark block

$$\frac{\partial \mathbf{h}^{[i_k]}(\cdot)}{\partial \vec{x}^{[k]}} = \mathbf{R}_t^T \rightarrow Z \times N_k = 2 \times 2$$

- Each 0 block is a 2×2 block, and we have $(K - 1)$ $\mathbf{0}$ blocks (only the i_k -th one is non-zero)

Note: $\mathbf{C}_t^{[i_k]}$ is a $2 \times (N_r + N_K)$ matrix.

In this example $N_r = 3$ and $N_K = 2K \rightarrow \mathbf{C}_t^{[i_k]}$ is a $2 \times 2K$ matrix

3.4.1 Measurement Noise

For each landmark we measure, we consider a Gaussian noise (in x and y). We assume it has zero mean with constant standard deviation.

$$\mathbf{n}_z \sim \mathcal{N}\left(\mathbf{n}_z; \mathbf{0}, \begin{bmatrix} \sigma_z^2 & 0 \\ 0 & \sigma_z^2 \end{bmatrix}\right)$$

Σ_z will be a diagonal matrix $2\bar{K} \times 2\bar{K}$

3.5 Data association

The observation vector $\bar{z}_t$ will originate from a subset of landmarks in the state.

The order of these measurements does not necessarily match the order of the landmarks in the state.

Let $j(i)$ be the index of the landmark that generates the i^{th} measurement block.

State representation

$\mathbf{x} = \mathbf{x}^{[r]}, \mathbf{x}^{[1]}, \dots, \mathbf{x}^{[K]}$ state = [robot; K landmarks]

Measurements

$\mathbf{z} = \mathbf{z}^{[1]}, \dots, \mathbf{z}^{[\bar{K}]}$ measurements ($\bar{K} \leq K$)

Measurement predictions

$\mathbf{h}(\mathbf{x}) = (h^{[1]}(\mathbf{x}), \dots, h^{[\bar{K}]}(\mathbf{x}))^\top$ $\bar{K}$ measurement predictions

Data association function

$j(i) \in [1, 2, \dots, K]$

$j(3) = 5$ means measurement 3 originates from landmark 5

For now we assume to know the assignment $j(i)$.

With this assignment we can build a prediction based on $\bar{K}$ measured landmarks, and its Jacobian!

$$\mathbf{h}(\mathbf{x}_t) = \begin{pmatrix} \mathbf{h}^{[j(1)]} \\ \mathbf{h}^{[j(2)]} \\ \vdots \\ \mathbf{h}^{[j(\bar{K})]} \end{pmatrix} \quad \mathbf{C}_t = \frac{\partial \mathbf{h}}{\partial \bar{\mathbf{x}}} = \begin{pmatrix} \frac{\partial \mathbf{h}^{[j(1)]}}{\partial \bar{\mathbf{x}}} \\ \frac{\partial \mathbf{h}^{[j(2)]}}{\partial \bar{\mathbf{x}}} \\ \vdots \\ \frac{\partial \mathbf{h}^{[j(\bar{K})]}}{\partial \bar{\mathbf{x}}} \end{pmatrix}$$

Note: C_t is a $(2\bar{K}) \times (N_r + N_K)$ matrix. Where $N_r = 3$ and $N_K = 2K$. Therefore C_t is a $(2\bar{K}) \times (3 + 2K)$

Example Partition the sensed landmarks in two classes:

- Already known landmarks (part of the state). Use them to perform an EKF correction
- New landmarks. Add them to the state after the EKF correction

3.6 Code

- Init values

```
1 [mu, sigma] = init()
2
3 id_to_state_map = ones(K, 1)*-1;
4 state_to_id_map = ones(K, 1)*-1;
```

- Predict

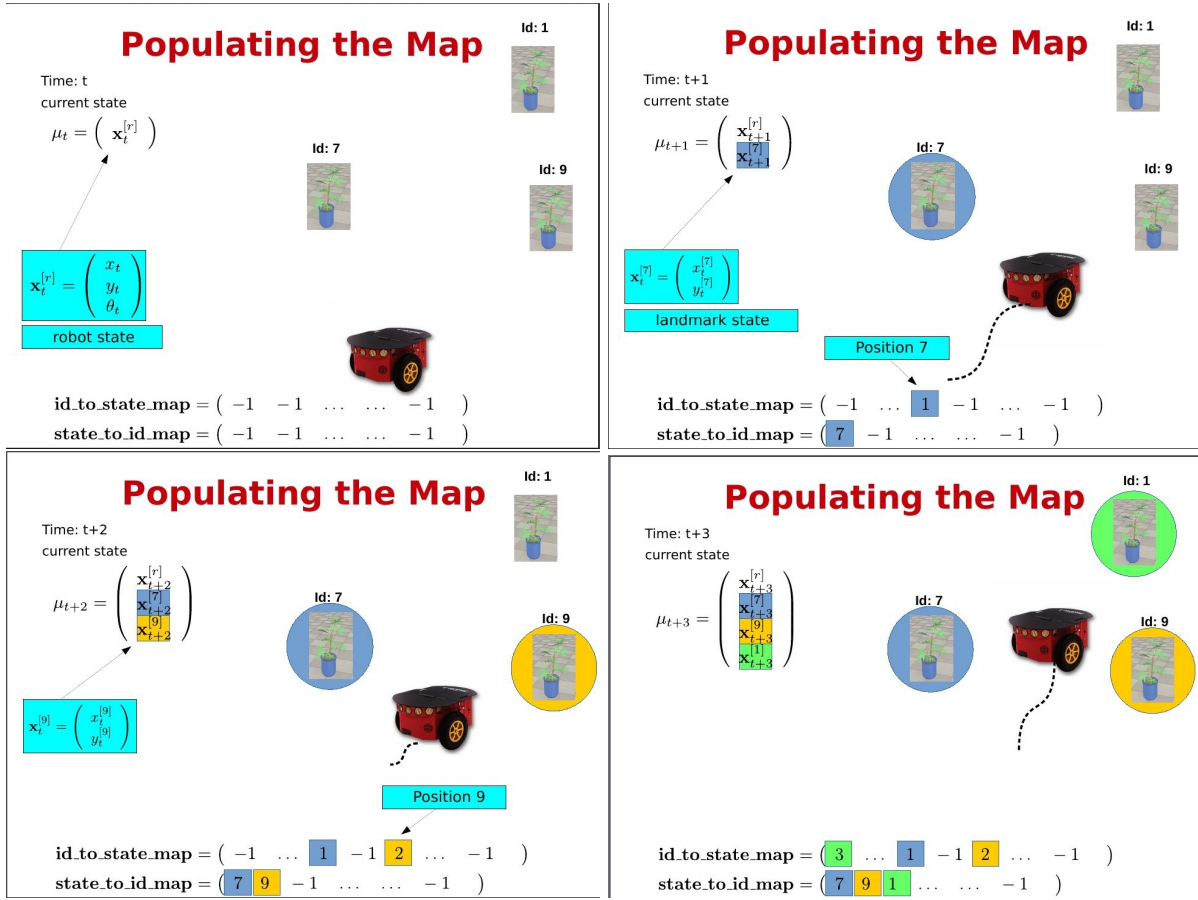


Figure 10: Populating the mapping

```

1 function [mu, sigma] = prediction(mu, sigma, control_input)
2
3 #domain spaces
4 dimension_mu = size(mu, 1);
5 dimension_u = 2;
6
7 #Jacobian A
8 #initialize A as an identity and fill only the robot block
9 A = eye(dimension_mu, dimension_mu);
10 A(1:3,1:3) = [1, 0, -u.x*sin(mu.theta);
11              0, 1, u.x*cos(mu.theta);
12              0, 0, 1];
13
14 #Jacobian B
15 #for each state variable we have to associate the available control
    inputs
16 B = zeros(dimension_mu, dimension_u);
17 B(1:3,:) = [cos(mu.theta), 0;
18            sin(mu.theta), 0;
19            0, 1];
20
21 #predict the robot motion with f(x,u)
22 #the transition model only affects the robot pose not the landmarks
23
24 #update the robot state
25
26 [mu.x; mu.y; mu.theta] = transition_model([mu.x; mu.y;
27 mu.theta], u);
28
29 #compose control noise covariance sigma_u
30 sigma_u = [noise^2+u_x^2, 0;
31           0, noise^2+ u_theta^2];
32
33 #predict sigma
34 sigma = A*sigma*A' + B*sigma_u*B';
35 end

```

- Update

```

1 function [mu, sigma, id_to_state_map, state_to_id_map] = correction(mu,
2 sigma, observations, id_to_state_map, state_to_id_map)
3
4 #determine how many landmarks we have seen in this step
5 M = length(observations);
6
7 #if I've seen no landmarks, i do nothing
8 if (M == 0)
9     return;
10 endif
11
12 #dimension of the state (robot pose + landmark positions)
13 dimension_state = size(mu,1);
14 mu_t = mu(1:2,:); # translational part of the robot pose
15 mu_theta = mu(3); # rotation of the robot
16
17 #rotation matrix
18 R = [cos(mu_theta) -sin(mu_theta);
19      sin(mu_theta) cos(mu_theta)];
20
21 #transposed rotation matrix
22 Rt = [cos(mu_theta), sin(mu_theta);
23       -sin(mu_theta) cos(mu_theta)];

```

```

24
25 #derivative of transposed rotation matrix
26 Rtp = [-sin(mu_theta),cos(mu_theta);
27         -cos(mu_theta),-sin(mu_theta)];
28
29 # for below computation, we need to count how many observations
30 # of old landmarks we have
31 number_of_known_landmarks = 0;
32
33 # Here two cases arise, the current landmark has been already seen, i.e.
34   REOBSERVED landmark,
35 # or the current landmark is completely new, i.e. NEW landmark.
36 #
37 # for simplicity we can divide the problem: first analyze only the
38   reobserved landmark
39 # and work with them as in a localization procedure (of course, with full
40   Jacobian now).
41 # With this reobserved landmark we compute the correction/update of mean
42   and covariance.
43 # Then, after the correction is done, we can simply add the new landmark
44   expanding the
45 # mean and the covariance.
46 #
47 # First of all we are interested in REOBSERVED landmark
48 z_t = [];
49 h_t = [];
50 C_t = [];
51 for i=1:M
52
53     #retrieve info about the observed landmark
54     measurement = observations(i);
55
56     #fetch the position in the state vector corresponding to the actual
57     measurement
58     n = id_to_state_map(measurement.id);
59
60     #IF current landmark is a REOBSERVED LANDMARK
61     if(n != -1)
62
63         #compute the index (vector coordinate) in the state vector
64         corresponding to the pose of the landmark;
65         id_state = 4+2*(n-1);
66
67         #increment the counter of observations originating
68         #from already known landmarks
69         number_of_known_landmarks++;
70
71         #add landmark measurement
72         z_t = [z_t; measurement];
73
74         #fetch the position of the landmark in the state (its x and y
75         coordinate)
76         landmark_mu=mu(id_state:id_state+1,:);
77
78         #where I predict i will see that landmark
79         delta_t = landmark_mu-mu_t;
80         measure_prediction = Rt* delta_t;
81
82         #add landmark measurement prediction
83
84         h_t = [h_t; measure_prediction];
85
86         #jacobian piece w.r.t. robot

```

```

79     C_r          = zeros(2, dimension_state);
80     C_r(1:2,1:2) = -Rt;
81     C_r(1:2,3)   = Rtp*delta_t;
82
83     #jacobian piece w.r.t. landmark
84     C_ik(:,id_state:id_state+1) = Rt;
85
86     #add jacobian piece to main jacobian
87     C_t = [C_t; C_ik];
88
89     endif
90 endfor
91
92 #if I have seen again at least one landmark
93 #I need to update
94 if ((number_of_known_landmarks > 0))
95
96     #observation noise
97     noise = 0.01;
98     sigma_z = eye(2*number_of_known_landmarks)*noise;
99
100    #Kalman gain
101    K = sigma * C_t'*(inv(C_t*sigma*C_t' + sigma_z));
102
103    #update mu
104    innovation = (z_t - h_t);
105    mu += K*innovation;
106
107    #update sigma
108    sigma *= (eye(dimension_state) - K*C_t);
109 endif
110 end

```

- Transition function

```

1 #Compute mu at time t given mu at time t-1
2 function mu_prime = transition_model(mu, u)
3     mu_prime(1) = mu.x + u.x*cos(mu.theta);
4     mu_prime(2) = mu.y + u.x*sin(mu.theta);
5     mu_prime(3) = mu.theta + u.w;
6 end

```

- At each simulation step

```

1 #Compute mu at time t given mu at time t-1
2 while(true)
3
4     %predict
5     [mu, sigma] = prediction(mu, sigma, u);
6
7     observations = getObservation(mu, sigma)
8
9     %correct
10    [mu, sigma, id_to_state_map, state_to_id_map] = correction(mu, sigma,
11        observations, id_to_state_map, state_to_id_map);
12
13    [mu, sigma, id_to_state_map, state_to_id_map] = addNewLandmarks(mu,
14        sigma, observations, id_to_state_map, state_to_id_map);
15
16 endfor

```

addNewLandmarks() will add the new detected landmarks to the state and to the covariance. Moreover it will update *id_to_state_map*, *state_to_id_map*

4 EKF SLAM with unknown data association

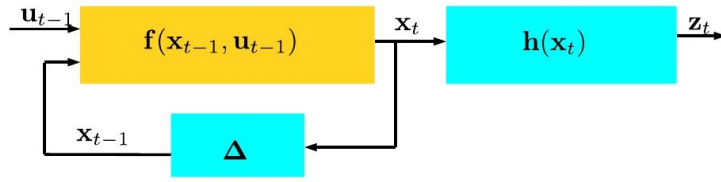


Figure 11: Dynamic schema

4.1 Assumptions

In the following we assume that

- Robot is controlled by translational and rotational velocities
- Senses a set of **not distinguishable** landmarks through a "2D landmark sensors"
- The **location** $l^{[i]}$ of the landmarks w.r.t. the world is **unknown**.

$$w_{l^{[i]}} = \begin{pmatrix} x^{[i]} \\ y^{[i]} \end{pmatrix} \in \mathbb{R}^2$$

- No prior knowledge on the map

Note: We want to develop a KF based algorithm to track the position of Orazio as it moves (localization) and, at the same time, the position of the observed landmarks (mapping) while performing data association.

4.2 Domain & Transition function & Control noise & Measurement function & Measurement noise

Exactly as previous exercise, but we do a slight change of notation: m becomes the m -th measurement while n becomes the n -th landmark.

4.3 Data association

We do **not** observe the landmark ids. When a new landmark appears, it's our duty to assign it a new, unique id.

We keep unchanged the state-id mapping structure

$$\begin{aligned} \text{id_to_state_map} &= \begin{pmatrix} -1 & -1 & \cdots & -1 \end{pmatrix} \\ \text{state_to_id_map} &= \begin{pmatrix} -1 & -1 & \cdots & -1 \end{pmatrix} \end{aligned}$$

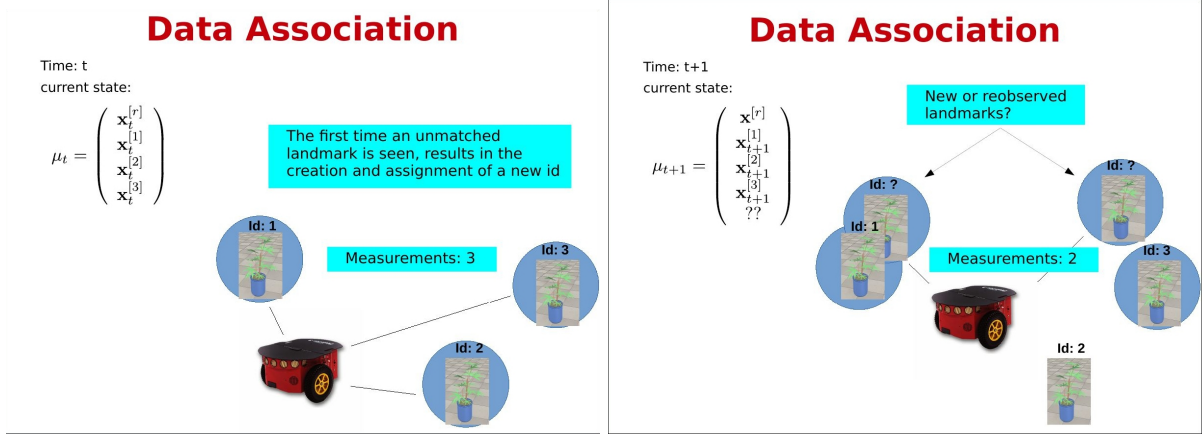


Figure 12: Populating the mapping

At each time step, compute the **likelihood** for each landmark/measurement pair:

$$a_{mn} = (\mathbf{z}^{[m]} - \mathbf{h}^{[n]}(\mathbf{x}_t))^T \boldsymbol{\Omega}_{n,n} (\mathbf{z}^{[m]} - \mathbf{h}^{[n]}(\mathbf{x}_t))$$

with information matrix (canonical):

$$\boldsymbol{\Omega}_{n,n} = \boldsymbol{\Sigma}_{n,n}^{-1}, \quad \boldsymbol{\Sigma}_{n,n} = \mathbf{C}_t^{[n]} \boldsymbol{\Sigma}_x (\mathbf{C}_t^{[n]})^T + \boldsymbol{\Sigma}_{z|x}$$

and assemble them in a **cost matrix**:

$$\mathbf{A} = \begin{pmatrix} a_{11} & a_{12} & a_{13} & \cdots & a_{1N} \\ a_{21} & a_{22} & a_{23} & \cdots & a_{2N} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ a_{M1} & a_{M2} & a_{M3} & \cdots & a_{MN} \end{pmatrix}$$

4.3.1 Pruning heuristics

Gating

- Choose a threshold τ_{accept}
- Extract the minimum for each row a_{mn}
- If $a_{mn} < \tau_{\text{accept}}$
 - *then* observation m is associated with landmark n
 - *otherwise*, m is a new landmark.

Note: Multiple measurements can be assigned to the same landmark

Best friends

- Take all the accepted associations from gating and check
- If

$$[a_{mn} = \min_m a_{mn}] \neq [a_{mn} = \min_n a_{mn}]$$

- *then* discard it
- *otherwise* keep the association

Note: Only one measurement can be assigned to the same landmark

Lonely best friends

- Define a small threshold $\gamma > 0$ and, for all surviving associations, extract the second best association for measurements $\hat{a}_m$ and landmarks $\hat{a}_n$ and check

- If

$$[\hat{a}_m - a_{mn} < \gamma] \quad \text{OR} \quad [\hat{a}_n - a_{mn} < \gamma]$$

- *then* discard it
- *otherwise* keep it

4.4 Code

- Init values

```
1 [mu, sigma] = init()
2
3 id_to_state_map = ones(K, 1)*-1;
4 state_to_id_map = ones(K, 1)*-1;
```

- **Predict:** as previous exercise
- **Correction:** as previous exercise
- **Transition function:** as previous exercise
- associateLandmarksIDS()

```
1 function observations = associateLandmarkIDs(mu, sigma, observations,
2     state_to_id_map)
3
4     state_dim = size(mu, 1);
5
6     # dimension of the current map (how many landmarks we have already seen
7     )
8     N = (state_dim-3)/2;
9
10    #if we do not any landmarks in the state to associate or have no
11    measurements
12    if(N == 0 || M == 0)
13        return;
14    endif
15
16    #readability
17    mu_landmarks = mu(4:end);
18    mu_robot      = mu(1:3);
19
20    #build the association matrix [(current observations) x (number of
21    landmarks)] - with maximum distance values
22    A = ones(M, N)*1e3;
23
24    #state iterator (we will use this to iterate over the landmarks in the
25    state)
26    id_state = 1;
27
28    #now we have to populate the association matrix
29    for n=1:N
30
31        #extract landmark from mu
32        mu_curr_landmark = mu_landmarks(id_state:id_state+1);
33
34        #retrieve measurement prediction
35        [measurement_prediction, C] = measurement_function(state_dim,
36            mu_robot, mu_curr_landmark, id_state);
```

```

31
32     #compute the covariance with noise
33     sigma_zx = eye(mu_curr_landmark)*0.01
34     #covariance for the current landmark
35     sigma_nn = C*sigma*C'+sigma_zx
36
37     #compute information matrix (canonical representation of sigma_nn)
38     omega_nn = inv(sigma_nn)
39
40     #for all landmarks
41     for m=1:M
42
43         #obtain current measurement
44         measurement = measurements(m);
45         z = [measurement.x_pose; measurement.y_pose];
46
47         #compute likelihood for this measurement and landmark
48         A(m, n) = (z-h)'*omega_nn*(z-h);
49     endfor
50
51     #move to the next landmark in the state
52     id_state += 2;
53 endfor
54
55 #now associate the measurement to the most promising landmark
56 # proposed associations will be stored in a [Mx3] matrix composed
57 # in this way
58 #
59 # [measurement id, proposed landmark id , association matrix value
60 #   a_mn]
61 #
62 # we will populate such a matrix with the associations surviving
63 # the gating heuristic for each step
64 #
65 # In the best friends and lonely best friend heuristics we will keep
66 # the association as is in case of success, otherwise we will put
67 # an id=0 to that measurment, meaning that the association is doubtful
68
69 #configuration heuristics
70 gating_tau = 1;
71 lonely_best_friend_gamma_threshold = 1e-3;
72
73 #1. Gating
74 for m=1:M
75
76     #return the min and index on the 'm-th' row
77     [a_mn, min_index] = min(A(m,:));
78
79     #if the association passes the gate
80     if(a_mn<gating_tau)
81
82         #add the possible association - creating the associations
83         #vector
84         # [measurement id, proposed landmark id , association matrix
85         #   value a_mn]
86         associations(end+1,:) = [m, min_index ,a_mn]
87     endif
88 endfor
89
90 #associations that survived the gating
91 number_of_gated_associations = size(associations, 1);
92
93 #2. Best friends

```

```

91     for i=1:number_of_gated_associations
92 2         a_mn = associations(i, 3);
93         proposed_landmark_id = associations(i, 2);
94
95         #compute column minimum
96         min_on_column = min(A(:, proposed_landmark_id));
97
98         #if the association is not the minimum in the column
99         if(min_on_column != a_mn)
100             #discard association, it is doubtful
101             associations(i, 2) = 0;
102         endif
103     endfor
104
105     #3. Lonely best friend
106     number_of_valid_associations = 0;
107     if(M > 1)
108         for i=1:number_of_gated_associations
109             a_mn = associations(i, 3);
110             measurement_id = associations(i, 1);
111             proposed_landmark_id = associations(i, 2);
112
113             #this association is doubtful, skip evaluation
114             if(proposed_landmark_id == 0)
115                 continue;
116             endif
117
118             #obtain second best(aka min) value of the row
119             ordered_row = unique(A(measurement_id,:));
120             second_row_min_value = ordered_row(2);
121
122             #obtain second best(aka min) value of the column
123             ordered_col = unique(A(:,proposed_landmark_id));
124             second_col_min_value = ordered_col(2);
125
126             #check if the association is ambiguous
127             if(a_mn-second_row_min_value<lonely_best_friend_gamma_threshold
128                 || a_mn-second_col_min_value<
129                 lonely_best_friend_gamma_threshold )
130
131                 #discard association, it is doubtful
132                 associations(i,2) = 0;
133             else
134                 #we have found a valid association!
135                 ++number_of_valid_associations;
136             endif
137         endfor
138     endif
139
140     #assign the associations to the observations
141     for i=1:number_of_gated_associations
142         measurement_id = associations(i, 1);
143         proposed_landmark_id = associations(i, 2);
144         ##assign the proposed association OR 0 (a magic number that
145         indicates no landmark was found)
146         observations.observation(measurement_id).id = proposed_landmark_id;
147     endfor
148 end

```

- At each simulation step

```

1 #Compute mu at time t given mu at time t-1
2 while(true)

```

```

3
4 %predict
5 [mu, sigma] = prediction(mu, sigma, u);
6
7 observations_t = associateLandmarkIDs(mu, sigma, observations(t),
8     state_to_id_map);
9
10 %correct
11 [mu, sigma, id_to_state_map, state_to_id_map] = correction(mu, sigma,
12     observations, id_to_state_map, state_to_id_map);
13
14 [mu, sigma, id_to_state_map, state_to_id_map] = addNewLandmarks(mu,
15     sigma, observations, id_to_state_map, state_to_id_map);
16
17 endfor

```

addNewLandmarks() will add the new detected landmarks to the state and to the covariance. Moreover it will update *id_to_state_map*, *state_to_id_map*

5 UKF localization

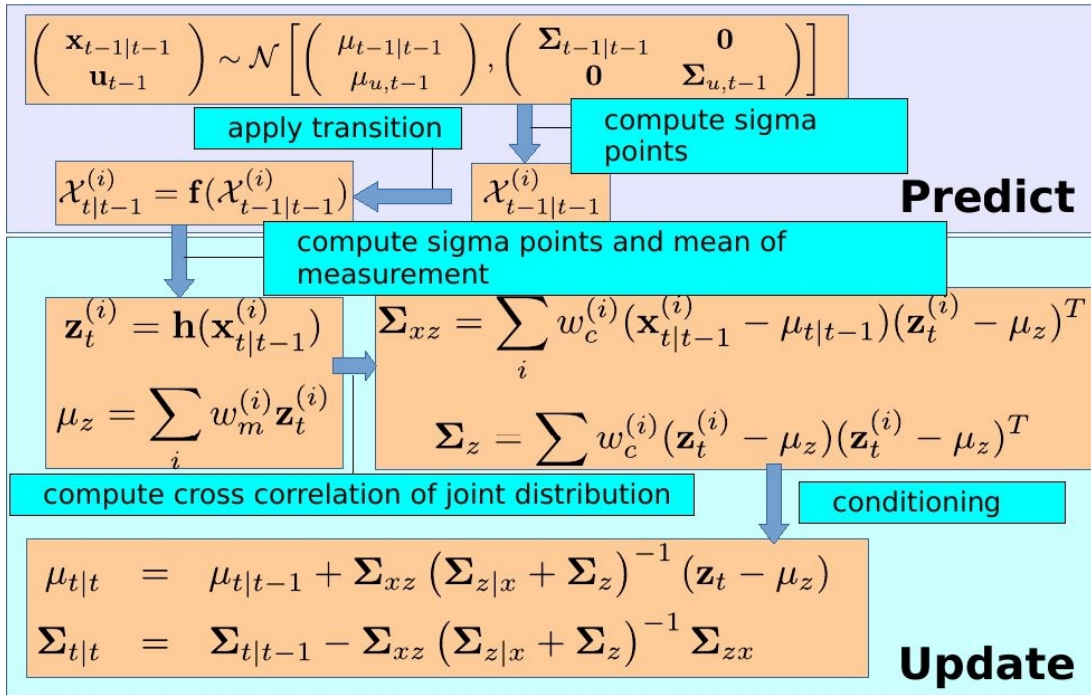


Figure 13: UKF filter wrapup

5.1 Assumptions

In the following we assume that

- Robot is controlled by translational and rotational velocities
- Senses a set of **uniquely distinguishable landmarks** through a “2D landmark sensor”
- The **location** $l^{[i]}$ of the landmarks w.r.t. the world is **known**.

$$w_{l^{[i]}} = \begin{pmatrix} x^{[i]} \\ y^{[i]} \end{pmatrix} \in \mathbb{R}^2$$

Note: We want to develop a UKF based algorithm to track the position of the robot as it moves (localization).

5.1.1 Domains

- State space (isometry)

$$\mathbf{X}_t = [\mathbf{R}_t | \vec{t}_t] \in SE(2) \rightarrow \vec{x}_t = \begin{pmatrix} x_t \\ y_t \\ \theta_t \end{pmatrix} \in \mathbb{R}^3$$

Note: Poses are not euclidean but we map them to 3D vectors

- Space of controls (inputs)

$$\vec{u}_t = \begin{pmatrix} \Delta_t v_t \\ \Delta_t \omega_t \end{pmatrix} = \begin{pmatrix} u_t^1 \\ u_t^2 \end{pmatrix} \in \mathbb{R}^2$$

- Space of observations (measurements)

$$\vec{z}_t^{[i]} = \begin{pmatrix} x_t^{[i]} \\ y_t^{[i]} \end{pmatrix} \in \mathbb{R}^2$$

5.2 Transition function & Control noise & Measurement function & Measurement noise

As in EKF localization

5.3 Code

- Init values

```
1 [mu, sigma] = init()
```

- computeSigmaPoints()

Recalling that:

Sigma Points



$$\begin{aligned} \lambda &= \alpha^2 n & w_m^{(0)} &= \frac{\lambda}{n + \lambda} \\ \mathbf{L} &= \sqrt{(n + \lambda) \mathbf{A}} & w_c^{(0)} &= w_m^{(0)} + (1 - \alpha^2 + \beta) \\ \mathbf{x}^{(0)} &= \mu & w_c^{(i)} &= w_m^{(i)} = \frac{1}{2(n + \lambda)} \\ \mathbf{x}^{(i)} &= \mu + [\mathbf{L}]_i \text{ for } i \in [1..n] \\ \mathbf{x}^{(i)} &= \mu - [\mathbf{L}]_{2n+1-i} \text{ for } i \in [n + 1..2n] \end{aligned}$$

Figure 14: Sigma points formulas

```

1 function [sigmaP, weightsM, weightsC] = compute_sigma_points(mu, sigma)
2
3     state_dim = size(mu,1);
4     num_of_sigma_points = 2*state_dim+1;
5
6     % From theory
7     k = 0;
8     alpha = 1e-3;
9     lambda = alpha^2 * state_dim;
10    bet = 2;
11
12    %init the sigma points matrix and its weight vectors
13    sigmaP = zeros(size(mu,1),num_of_sigma_points);
14    weightsM = zeros(num_of_sigma_points,1);
15    weightsC = zeros(num_of_sigma_points,1);
16
17    %% STEP 1: compute weights
18    % the first weights can be computed as
19    weightsM(1) = lambda/(state_dim+lambda);
20    weightsC(1) = weightsM(1) + (1 - alpha^2 + bet);
21
22
23    %% STEP 2: compute sigma points
24    % the first sigma point is the mean
25    sigmaP(:,1) = mu;
26
27
28    %compute the other weights
29    weight_value = 1/(2*(state_dim + lambda));
30    weightsM(2:num_of_sigma_points) = repmat(weight_value,
31        num_of_sigma_points-1, 1);
32    weightsC(2:num_of_sigma_points) = weightsM(2:num_of_sigma_points);
33
34    % chol extract the Cholesky Decomposition
35    % with the arg "lower" it returns the lower
36    % triangular matrix L, such that L'*L = A
37    [L] = chol(sigma*(state_dim+lambda) , "lower");
38
39    point_idx = 2;
40    for i=1:state_dim
41        L_i = L(:,i);
42        %half of the remaining 2*state_dim sigma points
43        sigmaP(:,point_idx) = mu + L_i;
44        point_idx++;
45        sigmaP(:,point_idx) = mu - L_i;
46        point_idx++;
47    end
48    endfunction

```

- Prediction

```

1 function [sigmaP, weightsM, weightsC] = prediction(mu, sigma, transition)
2
3     %motion noise
4     noise = 0.05;
5     v_noise = u.x^2;
6     w_noise = u.theta^2;
7
8     sigma_u = [ noise+u.x^2, 0;
9                 0, noise+u.theta^2];
10
11    sigma_xu = zeros(5,5);

```

```

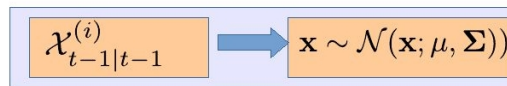
12     sigma_xu(1:3,1:3) = sigma;
13     sigma_xu(4:5,4:5) = sigma_u;
14
15     mu_xu = zeros(5,1);
16     mu_xu(1:3) = mu;
17     mu_xu(4:5) = [u_x; u_theta];
18
19     % extract the sigma points
20     [sigmaP_xu, weightsM, weightsC] = compute_sigma_points(mu_xu, sigma_xu)
21     ;
22     %the returned sigmaP matrix is composed by (2n+1) sigma point, one
23     % for each column
24
25     sigmaP = zeros(size(mu), 0);
26     %apply transition to every sigma point
27     for i=1:size(sigmaP_xu,2)
28         curr_sigma_mu = sigmaP_xu(1:3,i);
29         curr_input_u = [sigmaP_xu(4,i); 0; sigmaP_xu(5,i)];
30         % sigmaP(:,end+1) = motion_model(curr_sigma_mu, curr_input_u);
31         sigmaP = [sigmaP motion_model(curr_sigma_mu, curr_input_u)];
32     end
33 end

```

- reconstructMeanCov()

Recalling that:

Reconstruct Mean



$$\mu = \sum w_m^{(i)} \mathbf{x}^{(i)}$$

$$\Sigma = \sum w_c^{(i)} (\mathbf{x}^{(i)} - \mu)(\mathbf{x}^{(i)} - \mu)^T$$

Figure 15: Reconstruct mean formulas

```

1 function [mu, sigma] = reconstruct_mean_cov(sigmaP, wM, wC)
2
3     state_dim = size(sigmaP,1);
4     num_of_sigma_points = size(sigmaP,2);

```



```

5
6 %initialize mean
7 mu = zeros(state_dim,1);
8 %populate mean
9 for i=1:num_of_sigma_points
10     mu += wM(i)*sigmaP(:,i);
11 endfor
12
13 %initialize covariance
14 sigma = zeros(state_dim,state_dim);
15 %populate covariance
16 for i=1:num_of_sigma_points
17     delta = (sigmaP(:,i) - mu);
18     sigma += (delta)*(delta)' * wC(i);
19 endfor
20 end

```

- Update

```

1 function [mu, sigma] = correction(sigmaP, weightsM, weightsC, landmarks,
2     observations)
3
4 % determine how many landmarks we have seen
5 num_landmarks_seen = length(observations.observation);
6
7 % dimension of the state in dim, in our case is fixed to 3
8 state_dim = size(sigmaP,1);
9
10 %if I've seen no landmarks, i do nothing
11 if (num_landmarks_seen==0)
12     return;
13 endif
14
15 % init measurement vec
16 z_t = zeros(0, 1);
17 %retrieve the index of observed landmarks
18 id_x =1;
19 for i=1:num_landmarks_seen
20     %retrieve info about the observed landmark
21     measurement = observations.observation(i);
22
23     z_t = [z_t; measurement.x_pose; measurement.y_pose]; % where we see
24     landmark, stack on vec
25
26     current_land = searchById(landmarks, measurement.id);
27     lx = current_land.x_pose;
28     ly = current_land.y_pose;
29
30 %where I should see that landmark from each sigma point
31 for j=1:size(sigmaP,2)
32     current_sigma_point = sigmaP(:,j);
33     measure_prediction = measurement_function(current_sigma_point,
34         [lx; ly]);
35     predicted_sigma_z(id_x:id_x+1,j) = measure_prediction;
36 endfor
37 id_x = id_x + 2;
38
39 endfor
40
41 %once we have computed our predicted_sigma_z, we can reconstruct h_t
42 and relative covariace
43 [mu_z, Sigma_z] = reconstruct_mean_cov(predicted_sigma_z, weightsM,
44     weightsC);

```

```

40 % here we need to correct mu and sigma, so first of all we reconstruct
    mu and sigma from sigma points
41 [mu, sigma] = reconstruct_mean_cov(sigmaP, weightsM, weightsC);
42
43 %observation noise
44 noise = 0.01;
45 Sigma_noise = eye(2*num_landmarks_seen)*noise;
46
47 Sigma_xz = zeros(state_dim, 2*num_landmarks_seen);
48 for i=1:size(sigmaP,2)
49     delta_x=(sigmaP(:,i) - mu);
50     delta_z=predicted_sigma_z(:,i) - mu_z;
51     Sigma_xz += weightsC(i)*delta_x*delta_z';
52 end
53
54 %Kalman gain
55 K = Sigma_xz * inv(Sigma_z + Sigma_noise);
56
57 %update mu
58 error = (z_t - mu_z);
59 correction = K*error;
60 mu = mu + correction;
61
62 %update sigma
63 sigma = sigma - K*(Sigma_z + Sigma_noise)*K';
64
65
66 end

```

- At each simulation step

```

1 #predict
2 [sigmaP, wM, wC] = prediction(mu, sigma, transition_t);
3
4 #obtain observations
5 observations_t = observations(i);
6
7 #correct
8 [mu, sigma] = correction(sigmaP, wM, wC, landmarks, observations_t);

```

6 Particle localization

6.1 Assumptions

In the following we assume that

- Robot is controlled by translational and rotational velocities
- Senses a set of **indistinguishable landmarks** through a “2D landmark sensor”
- The **location** $l^{[i]}$ of the landmarks w.r.t. the world is **known**.

$$w_{l^{[i]}} = \begin{pmatrix} x^{[i]} \\ y^{[i]} \end{pmatrix} \in \mathbb{R}^2$$

Note: We want to develop a Particle filter based algorithm to track the position of the robot as it moves (localization).

6.1.1 Domains

- State space (isometry)

$$\mathbf{X}_t = [\mathbf{R}_t | \vec{t}_t] \in SE(2) \rightarrow \vec{x}_t = \begin{pmatrix} x_t \\ y_t \\ \theta_t \end{pmatrix} \in \mathbb{R}^3$$

Note: Poses are not euclidean but we map them to 3D vectors

- Space of controls (inputs)

$$\vec{u}_t = \begin{pmatrix} \Delta_t v_t \\ \Delta_t \omega_t \end{pmatrix} = \begin{pmatrix} u_t^1 \\ u_t^2 \end{pmatrix} \in \mathbb{R}^2$$

- Space of observations (measurements)

$$\vec{z}_t^{[i]} = \begin{pmatrix} x_t^{[i]} \\ y_t^{[i]} \end{pmatrix} \in \mathbb{R}^2$$

6.2 Transition function

As previous exercise

6.3 Control noise

Thanks to Particle Filter method, we can remove the Gaussian assumption. E.g. we assume here that the velocity inputs are affected by a uniform noise whose amplitude grows with the speed.

Translational and rotational noise are assumed independent.

$$\vec{n}_{u,t} \sim \begin{pmatrix} a_1 ||u_t^1|| U(-0.5, 0.5) \\ a_2 ||u_t^2|| U(-0.5, 0.5) \end{pmatrix}$$

6.4 Measurement noise

We assume the measurement noise is zero mean, Gaussian and constant

$$\vec{n}_z \sim \mathcal{N}\left(\vec{n}_z; 0, \begin{pmatrix} \sigma_z^2 & 0 \\ 0 & \sigma_z^2 \end{pmatrix}\right)$$

With this noise, the observation model becomes

$$p(\vec{z}_t^{[i]} | \vec{x}_t) \propto \exp\left(-(\mathbf{h}^{[i]}(\vec{x}_t) - \vec{z}_t^{[i]})^T \Sigma_z^{-1} (\mathbf{h}^{[i]}(\vec{x}_t) - \vec{z}_t^{[i]})\right)$$

$$p(\vec{z}_t | \vec{x}_t) = \prod_i p(\vec{z}_t^{[i]} | \vec{x}_t)$$

6.5 Code

- Init values

```
1 % how many sample we want to deal with
2 dim_samples = 1000;
3
4 % uniformly sample from free space
5 samples = sampleFromFreeSpace(map_min, map_max, dim_samples);
6
7 % init weights to 1
8 weights = ones(1, dim_samples);
```

- Predict

When a new control u_{t-1} becomes available, we compute the location by

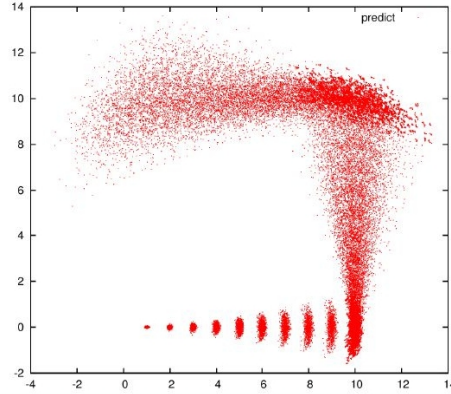
- generating an independent noise sample for each particle

$$\vec{n}_u^{(i)} \sim p(\vec{n}_u)$$

- computing the location of the new sample through the transition function, evaluated at the particle, at the control and at the noise sample

$$\vec{x}_{t|t-1}^{(i)} = \mathbf{f}(\vec{x}_{t-1|t-1}^{(i)}, \vec{u}_{t-1} + \vec{n}_u^{(i)})$$

$$w_{t|t-1}^{(i)} = w_{t-1|t-1}^{(i)}$$



evolution of the particles when the robot moves on a square

Figure 16: Square motion particle evolution

```

1 function samples = prediction(samples, transition)
2
3     dim_particles = size(samples,2);
4
5     a1 = abs(u.x);
6     a2 = abs(u.theta);
7     %apply transition to every particle
8     for i=1:dim_particles
9         %sample noise here and apply
10        noise.x = (rand() - 0.5)*a1;
11        noise.theta = (rand() - 0.5)*a2;
12
13        samples(:,i) = motion_model(samples(:,i), u + [noise.x; noise.theta
14        ]);
15    end
16 endfunction

```

- Update

To compute the update we need to address the data association

We solve it in a greedy manner, for each particle

- given a particle, we evaluate the likelihood of all combinations landmarks/measurements

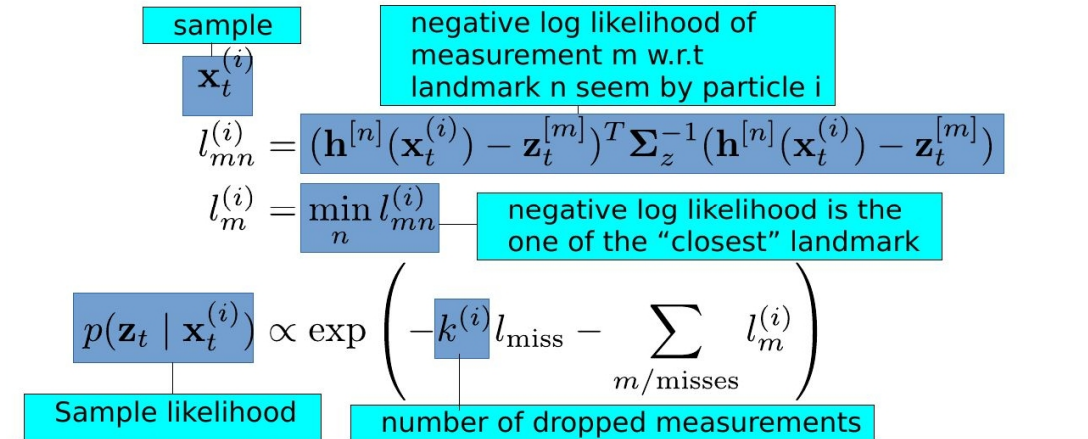


Figure 17: update help 1

Being able to evaluate the likelihood of each predicted sample we can perform the conditioning

$$w_{t|t}^{(i)} = w_{t|t-1}^{(i)} p(\mathbf{z}_t | \mathbf{x}_{t|t-1}^{(i)})$$

$$\mathbf{x}_{t|t}^{(i)} = \mathbf{x}_{t|t-1}^{(i)}$$

After conditioning we NEED to resample

this will be the distribution after resampling

$$\{j^{(i)}\} = \text{uniformSample}(\{w_{t|t}^{(i)}\})$$

$$\tilde{\mathbf{x}}_{t|t}^{(i)} = \mathbf{x}_{t|t}^{(j^{(i)})}$$

$$\tilde{w}_{t|t}^{(i)} = \frac{1}{\#\text{samples}}$$

Figure 18: update help 2

```

1 function weights = update(samples, weights, landmarks, observations)
2
3 % determine how many landmarks we have seen
4 num_landmarks_seen = length(observations);
5
6 num_particles = size(samples,2);
7
8 num_landmarks = size(landmarks,2);

```

```

9
10 %if I've seen no landmarks, i do nothing
11 if (num_landmarks_seen==0)
12     return;
13 endif
14
15 sigma_z_noise = 5;
16 Sigma_z_noise = [sigma_z_noise^2 0;
17                 0 sigma_z_noise^2];
18 Omega_z_noise = inv(Sigma_z_noise);
19
20 l_miss = 30;
21
22 for i=1:num_particles
23     l_mn = zeros(num_landmarks_seen, num_landmarks);
24     particle = samples(:,i);
25
26     for n=1:num_landmarks
27         landmark = landmarks(n);
28         land_x = landmark.x_pose;
29         land_y = landmark.y_pose;
30         [curr_h, _] = measurement_function(particle, [land_x; land_y]);
31
32         for m=1:num_landmarks_seen
33             measurement = observations.observation(m);
34             delta = curr_h - [measurement.x_pose; measurement.y_pose];
35             l_mn(m,n) = delta' * Omega_z_noise * delta;
36         endfor
37     endfor
38
39     negative_log_likelihood = min(l_mn');
40     dropped_measurements = 0;
41     for k=1:num_landmarks_seen
42         if(negative_log_likelihood(k) > l_miss)
43             negative_log_likelihood(k) = 0;
44             dropped_measurements++;
45         endif;
46     endfor
47
48     sum_of_negative_log_likelihood = sum(negative_log_likelihood);
49
50     weights(i) *= exp(-dropped_measurements*l_miss -
51                     sum_of_negative_log_likelihood);
52
53 endfor
endfunction

```

- resample()

```

1 function [new_samples, new_weights] = resample(samples, weights,
2         dim_samples)
3
4     indices = uniformSample(weights', dim_samples);
5
6     new_samples = samples;
7     new_weights = ones(1,dim_samples)/dim_samples;
8
9     for i=1:dim_samples
10         new_samples(:,i) = samples(:,indices(i));
11     endfor
12 endfunction

```

- At each simulation step

```

1 samples = prediction(samples, transitions(i));
2
3 weights = update(samples, weights, landmarks, observations(i));
4
5 [samples, weights] = resample(samples, weights, dim_samples);

```

7 Least squares application: Odometry Calibration

Algorithm (one Iteration)

Clear $\mathbf{H}$ and $\mathbf{b}$, aka assign the correct dimensions

$$\mathbf{H} \leftarrow 0 \quad \mathbf{b} \leftarrow 0$$

For each measurement, update $\mathbf{H}$ and $\mathbf{b}$

$$\mathbf{e}^{[i]} \leftarrow \mathbf{h}^{[i]}(\mathbf{x}^*) - \mathbf{z}^{[i]} \quad \leftarrow \text{Compute error}$$

$$\mathbf{J}^{[i]} \leftarrow \left. \frac{\partial \mathbf{e}^{[i]}(\mathbf{x})}{\partial \mathbf{x}} \right|_{\mathbf{x}=\mathbf{x}^*} \quad \leftarrow \text{Compute Jacobian}$$

$$\mathbf{H} \leftarrow \mathbf{H} + \mathbf{J}^{[i]T} \mathbf{\Omega}^{[i]} \mathbf{J}^{[i]}$$

$$\mathbf{b} \leftarrow \mathbf{b} + \mathbf{J}^{[i]T} \mathbf{\Omega}^{[i]} \mathbf{e}^{[i]}$$

Update the estimate with the perturbation

$$\Delta \mathbf{x} \leftarrow \text{solve}(\mathbf{H} \Delta \mathbf{x} = -\mathbf{b}) \quad \leftarrow \text{Solve linear system}$$

$$\mathbf{x}^* \leftarrow \mathbf{x}^* + \Delta \mathbf{x}$$

Figure 19: GN iteration

7.1 Assumption

In the following we assume that:

- We have a robot which moves in an environment, gathering the odometry measurements $\vec{u}_i$, affected by a systematic error.
- For each $\vec{u}_i$ we have a ground truth $\vec{u}_i^*$ provided us by an external sensor.
- There is a function $\mathbf{f}_i(\vec{x})$ (what we have to estimate) which, given some bias parameters $\vec{x}$, returns an unbiased odometry for the reading $\vec{u}_i'$ as follows

$$\vec{u}_i' = \mathbf{f}_i(\vec{x}) = \begin{pmatrix} x_{11} & x_{12} & x_{13} \\ x_{21} & x_{22} & x_{23} \\ x_{31} & x_{32} & x_{33} \end{pmatrix} \vec{u}_i$$

7.1.1 Domains

- State vector is

$$\vec{x} = (x_{11} \ x_{12} \ x_{13} \ x_{21} \ x_{22} \ x_{23} \ x_{31} \ x_{32} \ x_{33})^T \in \mathbb{R}^9$$

Namely the *unbiasing* transformation parameters

- Measurement

$$\vec{u}_i = \begin{pmatrix} u_{i,x} \\ u_{i,y} \\ u_{i,\theta} \end{pmatrix} \in \mathbb{R}^3$$

- The error function (or prediction function) is

$$e_i(\vec{x}) = \vec{u}_i^* - \begin{pmatrix} x_{11} & x_{12} & x_{13} \\ x_{21} & x_{22} & x_{23} \\ x_{31} & x_{32} & x_{33} \end{pmatrix} \vec{u}_i$$

With derivative

$$\mathbf{A}_i = \frac{\partial e_i(\vec{x})}{\partial \vec{x}} = - \begin{pmatrix} u_{i,x} & u_{i,y} & u_{i,z} & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & u_{i,x} & u_{i,y} & u_{i,z} & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & u_{i,x} & u_{i,y} & u_{i,z} \end{pmatrix}$$

7.2 Code

- Transform to vector

```
1 #computes the pose vector v from an homogeneous transform A
2 function v = t2v(A)
3     v(1:2,1) = A(1:2,3);
4     v(3,1) = atan2( -A(1,2),A(1,1))
5 end
```

- Vector to transform

```
1 # computes the homogeneous transform matrix A of the pose vector v
2 function A = v2t(v)
3
4     c = cos(v(3));
5     s = sin(v(3));
6     A = [c, -s, v(1);
7          s, c, v(2);
8          0, 0, 1];
9
10 end
```

- Compute odometry trajectory

```
1 #computes the trajectory of the robot by chaining up
2 #the incremental movements of the odometry vector
3 function T=compute_odometry_trajectory(U)
4     T=zeros(size(U,1),3);
5     current_T=v2t(zeros(1,3));
6     for i=1:size(U,1),
7         u=U(i,1:3)';
8         # this is P in the slides (thank bart)
9
10         # current_T is the absolute pose obtained by concatenating
11         # all the relative transformation until the current timestamp i
12         current_T *= v2t(u);
13         T(i,1:3)=t2v(current_T)';
14     end
15 end
```

- Apply odometry correction


```

1 #computes a calibrated vector of odometry measurements
2 #by applying the bias term to each line of the measurements
3 function C=apply_odometry_correction(X, U)
4     C=zeros(size(U,1),3);
5     for i=1:size(U,1),
6         u=U(i,1:3)';
7         uc = X*u;
8         C(i,:)=uc;
9     end
10 end

```

- Least squares calibrate odometry

```

1 #this function computes the error of the i^th measurement in Z
2 #given the calibration parameters
3 function e=error_function(i,X,Z)
4     ustar=Z(i,1:3)';
5     u=Z(i,4:6)';
6     e= ustar-X*u;
7 end
8
9 #derivative of the error function for the ith measurement in Z
10 #does not depend on the state
11 function A=jacobian(i,Z)
12     u=Z(i,4:6);
13     A=zeros(3,9);
14     A(1,1:3)= -u;
15     A(2,4:6)= -u;
16     A(3,7:9)= -u;
17 end
18
19 #this function solves the odometry calibration problem
20 #given a measurement matrix Z.
21 function X=ls_calibrate_odometry(Z)
22     H=zeros(9,9);
23     b=zeros(9,1);
24     #initial solution (the identity transformation)
25     X=eye(3);
26
27     for i=1:size(Z,1),
28         e=error_function(i,X,Z);
29         A=jacobian(i,Z);
30         H += A'*A;
31         b += A'*e;
32     end
33     #solve the linear system
34     deltaX=-H\b;
35     #this reshapes the 9x1 increment vector in a 3x3 atrix
36     dX=reshape(deltaX,3,3)';
37     #computes the cumulative solution
38     X=X+dX;
39 end

```

- Workflow.

Assume the calibration matrix contains for each row

$$(u_x^* \quad u_y^* \quad u_\theta^* \quad u_x \quad u_y \quad u_\theta)$$

Where both $\vec{u}^*$ and $\vec{u}$ are in relative coordinates, e.g. **motion of the robot between two consecutive frames.**

```

1 #load the calibration matrix

```

```

2 Z=load("/stuff");
3
4 #compute the ground truth trajectory
5 TrueTrajectory=compute_odometry_trajectory(Z(:,1:3));
6
7 #compute the uncalibrated odometry
8 OdomTrajectory=compute_odometry_trajectory(Z(:,4:6));
9
10 #compute the calibration parameters
11 X=ls_calibrate_odometry(Z);
12
13 C0dom=apply_odometry_correction(X,Z(:,4:6));
14
15 CalTrajectory=compute_odometry_trajectory(C0dom);

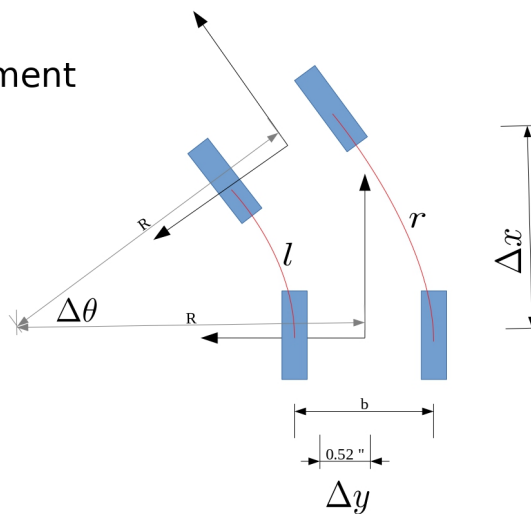
```

8 Least squares application: Unicycle odometry calibration

Differential Drive (Unicycle)

- r, l : motion on the ground wheels
- R : radius of curvature
- $\Delta x, \Delta y, \Delta\theta$: displacement from previous origin

$$\begin{aligned}
 r &= \Delta\theta \cdot (R + b/2) \\
 l &= \Delta\theta \cdot (R - b/2) \\
 \Delta\theta &= \frac{r - l}{b} \\
 R &= \frac{r + l}{2\Delta\theta} \\
 \Delta x &= R \sin(\Delta\theta) \\
 \Delta y &= R(1 - \cos(\Delta\theta))
 \end{aligned}$$



Differential Drive (Unicycle)

Angles usually small, remove singularity around $\Delta \theta = 0$ by Taylor expansion (up to 4th order)

$$\begin{aligned}\frac{\sin \theta}{\theta} &\simeq \overbrace{1 - \frac{\theta^2}{6} + \frac{\theta^4}{120} - \frac{\theta^6}{5040}}^{P_x(\theta)} \\ \frac{1 - \cos \theta}{\theta} &\simeq \overbrace{\frac{\theta}{2} - \frac{\theta^3}{24} + \frac{\theta^5}{720}}^{P_y(\theta)}\end{aligned}$$

Differential Drive (Unicycle)

Angles usually small, remove singularity around $\Delta \theta = 0$ by Taylor expansion (up to 4th order)

$$\begin{aligned}\frac{\sin \theta}{\theta} &\simeq \overbrace{1 - \frac{\theta^2}{6} + \frac{\theta^4}{120} - \frac{\theta^6}{5040}}^{P_x(\theta)} \\ \frac{1 - \cos \theta}{\theta} &\simeq \overbrace{\frac{\theta}{2} - \frac{\theta^3}{24} + \frac{\theta^5}{720}}^{P_y(\theta)}\end{aligned}$$

Differential Drive

We can rewrite the stuff as

$$\begin{aligned}
 \Delta_+^{[i]} &= r^{[i]} + l^{[i]} \\
 \Delta_-^{[i]} &= r^{[i]} - l^{[i]} \\
 \Delta\theta^{[i]} &= \frac{\Delta_-^{[i]}}{b} \\
 \Delta x^{[i]} &= \frac{\Delta_+^{[i]}}{2} P_x(\Delta\theta^{[i]}) \\
 \Delta y^{[i]} &= \frac{\Delta_+^{[i]}}{2} P_y(\Delta\theta^{[i]})
 \end{aligned}$$

Figure 20: Unicycle model

8.1 Assumptions

In the following we assume that

- We are given the relative encoder measures $t_r^{[i]}, t_l^{[i]}$
- We are given the robot pose on the plane from an external measurement system
- We are given with *sloppy nominal* value of parameters for k_l, k_r, b (what we have to estimate)

8.1.1 Domains

- State vector is

$$\vec{x} = \begin{pmatrix} k_r \\ k_l \\ b \end{pmatrix} \in \mathbb{R}^3$$

- Measurement

$$\vec{z}^{[i]} = \begin{pmatrix} t_r^{[i]} \\ t_l^{[i]} \end{pmatrix} \in \mathbb{R}^2$$

- Error function

$$\mathbf{h}^{[i]}(\vec{x}) = \text{odom}(\vec{x}, \vec{z}^{[i]}) - \vec{u}^{*[i]}$$

where *odom* is a function that takes the unicycle parameters and the encoder ticks and return the odometry (the displacement) for the current observation.

Note: We do not compute an explicit Jacobian for $h(\vec{x})$, we compute a numerical one.

8.2 Code

- $t2v(A)$, $v2t(v)$ and `compute_odometry_trajectory(U)` as before
- Odom

```
1 #Takes ticks and unicycle parameters and returns the odometry
2 function delta = odom(x,ticks)
3     kl = x(1);
4     kr = x(2);
5     b  = x(3);
6     tl = ticks(1);
7     tr = ticks(2);
8     dl = tl*kl;
9     dr = tr*kr;
10    dth = (dr-dl)/b;
11    d_plus2 = 0.5 * (dr+dl);
12    dx = d_plus2 * sin(dth)/dth;
13    dy = d_plus2 * (1-cos(dth))/dth;
14    delta = [dx, dy, dth]';
15 endfunction
```

- Least squares unicycle calibrate odometry

```
1 function [e, J] = errorAndJacobian(x,z)
2     ticks = z(1:2);
3     meas  = z(3:5);
4
5     pred  = h_odom(x,ticks);
6     e     = pred-meas;
7     J     = zeros(3,3);
8     epsilon = 1e-3;
9     inv_eps2= 0.5/epsilon;
10    for (i=1:3)
11        e_vec = zeros(3);
12        e_vec(i)=epsilon;
13        #Numeric jacobian (incremental limit ratio)
14        J(:,i) = inv_eps2*(h_odom(x+e_vec, ticks) -h_odom(x-e_vec, ticks));
15    endfor;
16 endfunction
17
18 function [x_new, chi] = ls_calibrate_odometry_unicycle (x, Z)
19
20 H=zeros(3,3);
21 b=zeros(3,1);
22 nmeas=size(Z,2);
23 chi=0;
24
25 for (i = 1:nmeas)
26     #Do nothing if no motion
27     if all(Z(1:2,i) == [0 0])
28         continue
29     end
30     [e,J]=errorAndJacobian(x,Z(:,i));
31     H+=J'*J;
32     b+=J'*e;
33     chi+=e'*e;
34
35 endfor
36 dx=-H\b;
37 x_new = x+dx;
38 end
```

- Workflow.

Assume the calibration matrix contains for each row

$$(t_l \quad t_r \quad u_x^* \quad u_y^* \quad u_\theta^*)$$

Where both t_r and t_l are relative, e.g. **motion of the robot between two consecutive frames**.

```

1 Z=load("/stuff");
2
3 #compute the ground truth trajectory
4 TrueTrajectory=compute_odometry_trajectory(Z(:,3:5));
5
6 #Compute uncalibrated odometry matrix from all tick measurements
7 odom = stack_odometry(nominal_params,Z(:,1:2));
8 #Compute the uncalibrated odometry trajectory
9 OdomTrajectory=compute_odometry_trajectory(odom);
10
11 #Compute the calibration parameters
12 [X,chi]=ls_calibrate_odometry_unicycle (nominal_params,Z');
13
14 #Compute odometry matrix from all tick measurements with calibrated
    parameters
15 COdom = stack_odometry(X,Z(:,1:2));
16 #Compute the calibrated odometry trajectory
17 CalTrajectory=compute_odometry_trajectory(COdom);

```

9 Registration on a manifold

9.1 Assumptions

In the following we assume that

- We have points in the world, measurements in the robot frame and roughly known correspondences

9.1.1 Domains

- State

$$\mathbf{X} = \begin{pmatrix} \mathbf{R} & \vec{t} \\ 0 & 1 \end{pmatrix} \in SE(3)$$

- Perturbation

$$\Delta \vec{x} = (\Delta x \quad \Delta y \quad \Delta z \quad \Delta \alpha_x \quad \Delta \alpha_y \quad \Delta \alpha_z)^T = (\Delta \vec{t} \quad \Delta \vec{\alpha})^T$$

- Boxplus

$$\mathbf{X} \boxplus \Delta \vec{x} = v2t(\Delta \vec{x})\mathbf{X} = (\mathbf{R}(\Delta \vec{\alpha}) \mathbf{R} \quad \mathbf{R}(\Delta \vec{\alpha}) \vec{t} + \Delta \vec{t})$$

- Measurements

Note: Our Measurements are direction vectors of points expressed in the reference frame of the sensor

$$\mathbf{z} \in S^2 \xrightarrow{\text{eucl.repr.}} \begin{pmatrix} x \\ y \\ z \end{pmatrix}^T$$

s.t. $\|z\| = 1$

Where S^2 is the sphere (hence, a manifold).

- Boxminus

$$\mathbf{z}_1 \boxminus \mathbf{z}_2 = \Delta \vec{z} = (\theta \quad \phi)$$

i.e. relative angle on azimuth and elevation.

- Measurement function

$$\mathbf{h}^{[i]}(\mathbf{X}) = \frac{\mathbf{R}\vec{p}^{[i]} + \vec{t}}{\|\mathbf{R}\vec{p}^{[i]} + \vec{t}\|}$$

9.1.2 Measurement simplifications

Measurements in this form are complicated. To solve the problem with this formulation we should:

- define a rotation matrix $\mathbf{R}(\mathbf{z}) = \mathbf{R}(\theta)\mathbf{R}(\phi)$
- compute such a rotation matrix for both $\mathbf{z}_1$ and $\mathbf{z}_2$
- compute the rotation difference as $\mathbf{R}_d = \mathbf{R}_2^T * \mathbf{R}_1$
- extract azimuth and elevation from rotation difference

$$(\text{atan}_2(-\mathbf{R}_d(1, 2), \mathbf{R}_d(2, 2)), \text{atan}_2(-\mathbf{R}_d(3, 1), \mathbf{R}_d(3, 3)))$$

Therefore we simplify relaxing the condition to compute the distance on a geodesic and we compute the more easy euclidean distance. Basically we consider $\mathbf{z}$ with its euclidean representation.

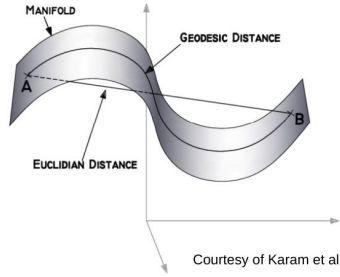


Figure 21: Euclidean vs geodesic distance

In this simplified formulation

- measurement function

$$\mathbf{h}^{[i]}(\mathbf{X}) = \text{normalize}(\mathbf{R}\vec{p}^{[i]} + \vec{t}) = \text{normalize}(\vec{p}^{[i]})$$

Therefore

$$\mathbf{h}^{[i]}(\mathbf{X} \boxplus \Delta \mathbf{x}) = \text{normalize}(\Delta \mathbf{R} \vec{p}^{[i]} + \Delta \vec{t})$$

- error function

$$\mathbf{e}^{[i]}(\mathbf{X}) = \mathbf{h}^{[i]}(\mathbf{X}) - \vec{z}^{[i]}$$

Note: Since now measurements are euclidean, no $\boxplus$ is needed

To compute the error function jacobian we need to apply the chain rule. Recalling that

•

$$\text{normalize}(\vec{v}) = \frac{\vec{v}}{\sqrt{\vec{v}^T \vec{v}}} = \frac{\vec{v}}{\|\vec{v}\|}$$

•

$$\frac{\partial \text{normalize}(\vec{v})}{\partial \vec{v}} = \frac{\mathbf{I}\|\vec{v}\| - \frac{\vec{v}\vec{v}^T}{\|\vec{v}\|}}{\|\vec{v}\|^2} = \frac{1}{\|\vec{v}\|}\mathbf{I} - \frac{1}{\|\vec{v}\|^3}\vec{v}\vec{v}^T$$

We can compute

$$\begin{aligned} \left. \frac{\partial \mathbf{e}^{[i]}(\mathbf{X} \boxplus \Delta \vec{x})}{\partial \Delta \vec{x}} \right|_{\Delta \vec{x}=0} &= \left. \frac{\partial \text{normalize}(\vec{p})}{\partial \vec{p}} \right|_{\vec{p}=\vec{p}^{[i]}} \cdot \left. \frac{\partial \mathbf{h}^{[i]}(\mathbf{X} \boxplus \Delta \vec{x})}{\partial \Delta \vec{x}} \right|_{\Delta \vec{x}=0} = \\ &= \frac{1}{\|\vec{p}^{[i]}\|^3} \left(\|\vec{p}^{[i]}\|^2 \mathbf{I} - \vec{p}^{[i]} \vec{p}^{[i]T} \right) \left(\mathbf{I} - [\vec{p}^{[i]}]_{\times} \right) \end{aligned} \quad (1)$$

Note: $\mathbf{J}$ is a 3×6 matrix ☺

9.2 Code

- Gauss Newton

```
1 function [X , chi_square]= doICP ( X, P, measurements , omega,
   num_iterations)
2
3 # ICP loop
4 for i=1:num_iterations
5     # initialize the quantities of the inner loop
6     chi_iteration = 0;
7     H = zeros(6,6);
8     b = zeros(6,1);
9     for j=1:num_points
10         # compute error and jacobian and update approximate hessian and
           gradient
11         measurement = measurements(:,j);
12         point = P(:,j);
13         [e,J] = errorAndJacobianNormRegistration(X,point,measurement);
14         # approximate hessian of e'* omega * e
15         H += J'*omega*J;
16         # gradient of e'* omega * e
17         b += J'*omega*e;
18         # take track of the chi square
19         chi_iteration += e'*e;
20     endfor
21     # update the current estimate
22     chi_square(i) = chi_iteration;
23     delta_x = -H\b;
24     rank(H)
25     X = v2t(delta_x) * X;
26     chi_iteration
27 endfor
```

- test

```
1 # initialize a random set of points
2 moving_cloud = 100 * randn(3,100);
3
4 # ground truth pose that we want to estimate
5 euclidian_pose = [0.10,-0.5,0.01,pi/8,-pi/8,pi/8]';
6 %euclidian_pose = [0,0,0,0,0,0]';
7
8 ground_truth_pose = v2t(euclidian_pose);
9
10 # generate measurements
11 fixed_cloud = ground_truth_pose(1:3,1:3) * moving_cloud + ground_truth_pose
   (1:3,4);
12
13 squared_cloud=fixed_cloud.*fixed_cloud;
14 norms=sqrt(sum(fixed_cloud.*fixed_cloud));
15 inv_norms=1./norms;
16 measurements = fixed_cloud.*inv_norms;
17
18 # we don't have any prior knowledge about the uncertainty of individual
   points
19 omega = eye(3);
20
21 num_points = size(fixed_cloud,1);
22
23 # initial guess for the pose
24 X = eye(4);
25
26 num_iterations = 10;
```



```

27 chi_square = zeros(num_iterations,1);
28 [X,chi_square] = doICP(X, P, measurements, omega, num_iterations);

```

10 Registration on a manifold with similarity

10.1 Assumptions

In the following we assume that

- We have points in the world, measurements in the robot frame and roughly known correspondences
- Measurements point are affected by a scaling factor

10.1.1 Domainis

- State vector

$$\mathbf{X} = \begin{pmatrix} \mathbf{R} & \vec{t} \\ 0 & s \end{pmatrix} \in SIM(3)$$

- Perturbation

$$\Delta \vec{x} = (\Delta x \quad \Delta y \quad \Delta z \quad \Delta \alpha_x \quad \Delta \alpha_y \quad \Delta \alpha_z \quad \Delta s)^T = (\Delta \vec{t} \quad \Delta \vec{\alpha} \quad \log s)^T$$

- Boxplus

We define $v2s(\Delta \vec{x})$ (vector to similarity) as

$$v2s(\Delta \vec{x}) = \begin{pmatrix} \mathbf{R}(\Delta \alpha) & \Delta \vec{t} \\ 0 & \exp(s) \end{pmatrix}$$

Therefore

$$\mathbf{X} \boxplus \Delta \vec{x} = v2s(\Delta \vec{x}) \mathbf{X}$$

- Measurements

$$\vec{z} \in \mathbb{R}^3$$

- Measurement function

$$\mathbf{h}^{[i]}(\mathbf{X} \boxplus \Delta \vec{x}) = (\mathbf{X} \boxplus \Delta \vec{x}) \vec{p}^{[i]} = \exp(\Delta s) \left(\mathbf{R}(\Delta \alpha) s \mathbf{R} \vec{p}^{[i]} + \vec{t} \right) + \Delta \vec{t}$$

Operations Given the previous domain, we define the following operations

—

$$\mathbf{X} \cdot \vec{p} = s \mathbf{R} \vec{p} + \vec{t}$$

—

$$\mathbf{X}_1 \mathbf{X}_2 = \begin{pmatrix} \mathbf{R}_1 \mathbf{R}_2 & \mathbf{R}_1 \vec{t}_2 + s_2 \vec{t}_1 \\ 0 & s_1 s_2 \end{pmatrix}$$

- Error function

$$\mathbf{e}^{[i]}(\mathbf{X} \boxplus \Delta \vec{x}) = \mathbf{h}^{[i]}(\mathbf{X} \boxplus \Delta \vec{x}) - \vec{z}^{[i]}$$

The jacobian will be

$$\begin{aligned} \mathbf{J}^{[i]} &= \frac{\partial \mathbf{e}^{[i]}(\mathbf{X} \boxplus \Delta \vec{x})}{\partial \Delta \vec{x}} \bigg|_{\Delta \vec{x}=0} = \left(\frac{\partial \mathbf{e}^{[i]}(\cdot)}{\partial \Delta \vec{t}} \quad \frac{\partial \mathbf{e}^{[i]}(\cdot)}{\partial \Delta \alpha} \quad \frac{\partial \mathbf{e}^{[i]}(\cdot)}{\partial \Delta s} \right) \bigg|_{\Delta \vec{x}=0} = \\ &= (\mathbf{I} \quad -[\mathbf{p}'^{[i]}]_{\times} \quad \mathbf{p}'^{[i]}) \end{aligned}$$

10.2 Code

- Gauss Newton

```
1 function [e,J]=errorAndJacobian(X,p,z)
2     t=X(1:3, 4);
3     R=X(1:3, 1:3);
4     s=X(4,4);
5
6     z_hat=s*(R*p+t);
7     e=z_hat-z;
8
9     J=zeros(3,7);
10    J(1:3,1:3)=eye(3);
11    J(1:3,4:6)=skew(-z_hat);
12    J(1:3,7)=z_hat;
13 endfunction
14
15 function [X, chi_stats, num_inliers]= doICP(x_guess, P, Z, num_iterations,
16     damping, kernel_threshold)
17 X=v2s(x_guess);
18 chi_stats=zeros(1,num_iterations);
19 num_inliers=zeros(1,num_iterations);
20 for (iteration=1:num_iterations)
21     H=zeros(7,7);
22     b=zeros(7,1);
23     chi_stats(iteration)=0;
24     for (i=1:size(P,2))
25         [e,J] = errorAndJacobian(X, P(:,i), Z(:,i));
26         chi = e'*e;
27
28         if (chi > kernel_threshold)
29             e*= sqrt(kernel_threshold/chi);
30             chi = kernel_threshold;
31         else
32             num_inliers(iteration)++;
33         endif;
34
35         chi_stats(iteration) += chi;
36
37         H+= J'*J;
38         b+= J'*e;
39     endfor
40
41     H+= eye(7)*damping;
42     dx = -H\b;
43     X = v2s(dx)*X;
44 endfunction
```

- test

```
1 #generate a bunch of sample points
2
3 n_points=100;
4 P_world=(rand(3,n_points)-0.5)*10;
5 inv_scaling = 0.5;
6
7 #ideal position of world w.r.t robot
8 x_true=[10, 0.2, 0.5, -pi/2, pi/2, -pi, log(inv_scaling)]';
9 X_true=v2s(x_true);
10
11 #compute the measurements by mapping them in the observer frame
12 P_world_hom=ones(4, n_points);
```

```

13 P_world_hom(1:3, :)=P_world;
14 Z_hom=X_true*P_world_hom;
15 Z=Z_hom(1:3,:).*Z_hom(4,:);
16
17 p_fix = P_world_hom(:,1)
18 hom_p = Z_hom(:, 1)
19 p = Z(:, 1)
20
21 noise_sigma=1;
22 # add some white noise
23 # Z(1:3,:) += normrnd(0, noise_sigma, 3, n_points);
24
25 iterations=100;
26 damping=0.6; # damping factor
27
28 chi_stats=zeros(1,iterations);
29 inliers_stats=zeros(1,iterations);
30
31 # test with a good initial guess
32 #x_guess=x_true+[-4,0.7,-0.7,0.4,0.4,0.1]'; #good initial guess
33 x_guess=[0,0,0,0,0,0,0]'; #bad initial guess
34 [X_result, chi_stats, inliers_stats] = doICP(x_guess, P_world, Z,
        iterations, damping, 1e6);

```